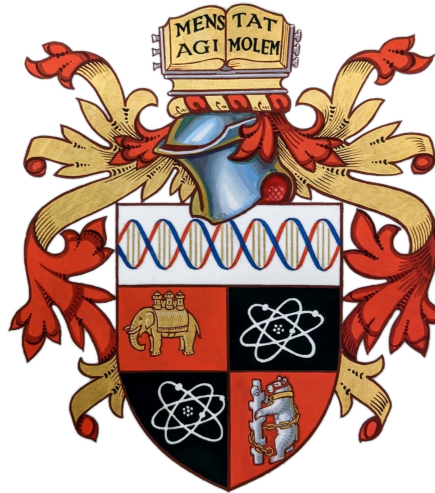




ALLOCATING BOUNDED LM
COGNITION IN PERSISTENT NPC
WORLDS: ADMISSION POLICIES AND
MEMORY STRATEGIES FOR
GROUNDED ACTION

by

DONG, WEI

This dissertation is submitted in partial fulfilment

of the requirements for the degree of

MASTER OF SCIENCE IN GAMES
ENGINEERING

UNIVERSITY OF WARWICK

WARWICK MANUFACTURING GROUP (WMG)

August 2026

CONTENTS

Contents	ii
List of figures	v
List of tables	vi
Declaration	vii
Abstract	viii
List of abbreviations	ix
1 Introduction	1
1.1 Background and Problem	1
1.2 Research Gap and Motivation	2
1.3 Research Questions and Objectives	2
1.4 Contributions	3
1.5 Scope and Delimitations	4
2 Literature Review	5
2.1 Limited LLM Cognition and Inference Budgets	5
2.2 NPC and Multi-Agent Decision Scheduling	6
2.3 Long-Term Memory Representation	7
2.4 Authoritative Persistent-World Simulation	8
2.5 Literature Synthesis and Research Gap	8
3 System Design and Implementation	10
3.1 System Architecture	10
3.2 Cognitive LOD and Decision Flow	10
3.2.1 Autonomy Decision Flow	10
3.2.2 Dialogue Routing Strategy	11
3.3 Authority and Validation	12
3.4 Persistence and Memory	14
3.5 Living Town Demo	14

4	Research Methodology	16
4.1	Overall Experimental Design	16
4.2	RQ1 Design	16
4.3	RQ2 Design	17
4.4	Measures and Outcome Definitions	19
4.5	Statistical Analysis	20
4.6	Data Collection and Run Integrity	21
4.7	Supplementary Evaluation of Cognitive LOD	21
4.7.1	Local L1 Model Quality and Cost Comparison	22
4.7.2	Sixty-Day Cognitive LOD Workload	22
4.7.3	Dialogue Tiering Evaluation	23
5	Results	25
5.1	Data Integrity and Sample Completion	25
5.2	RQ1 Results	26
5.2.1	Admission Overlap and Block-Level Results	27
5.2.2	Failure-Stage Decomposition	27
5.2.3	Provider Usage and Terminal Outcomes	28
5.3	RQ2 Results	28
5.3.1	Results under Different History Loads	29
5.3.2	Context and Provider Usage	30
5.3.3	Failures and Retries	30
5.4	Cognitive LOD and Local L1 Evaluation	31
5.4.1	Local L1 Quality and Cost	31
5.4.2	Sixty-Day Workload Results	32
5.4.3	Dialogue Tiering Results	33
6	Discussion	35
6.1	Interpretation of RQ1	35
6.2	Interpretation of RQ2	36
6.3	Combined System Implications	37
6.3.1	A ResolutionReadiness Signal for Admission	37
6.4	Practical Value of L0 and Local L1	38
6.5	Threats to Validity	39
6.6	Limitations and Future Work	40
7	Conclusion	41
7.1	Research Summary	41
7.2	Answers to the Research Questions	41
7.3	Supplementary Cognitive LOD Findings	42
7.4	Final Remarks	42

Bibliography	44
A Supplementary Cognitive LOD Materials	46
A.1 Lifecycle Log Field Dictionary	46
A.2 Controlled Dialogue Scenario List	47
A.3 Representative Lifecycle Records	48
B Proposed ResolutionReadiness Design	49
C Reproducibility and Supplementary Resources	52
C.1 Source Code	52
C.2 Demonstration Video	52
C.3 Research Artifacts	52

LIST OF FIGURES

3.1	Runtime architecture and authoritative state boundaries.	11
3.2	Cognitive LOD routing and grounded L2 execution.	13
3.3	L0–L1–L2 cognition routing in the Living Town Demo.	15
4.1	RQ1 and RQ2 experimental separation.	19

LIST OF TABLES

4.1	RQ2 problem strata.	18
4.2	RQ2 history-load tiers.	18
4.3	Local–remote L1 comparison cases.	22
4.4	Controlled dialogue routing cases.	23
5.1	Core dataset completion and integrity.	25
5.2	RQ1 outcomes and fairness diagnostics.	26
5.3	RQ1 results by top-four admission-overlap tier.	27
5.4	RQ1 terminal outcomes among admitted L2 sessions.	27
5.5	RQ1 Provider usage and terminal outcomes.	28
5.6	RQ2 quality and context reduction.	29
5.7	RQ2 outcomes by history-load tier.	29
5.8	RQ2 Provider usage by memory representation.	30
5.9	Local L1 decision quality.	31
5.10	Local L1 outcomes by expected route.	31
5.11	Local and remote L1 Provider usage.	32
5.12	Sixty-day Cognitive LOD workload.	32
5.13	Natural dialogue workload.	33
5.14	Controlled dialogue routing outcomes.	34
A.1	Dialogue lifecycle fields.	46
A.2	Controlled Cognitive LOD dialogue scenarios.	47
B.1	ResolutionReadiness states.	50

DECLARATION

This dissertation is submitted in partial fulfilment of the requirements for the degree of Master of Science in Games Engineering. It has been composed by myself and has not been submitted in any previous application for any degree. The work presented (including data generated and data analysis) was carried out by the author.

ABSTRACT

Persistent non-player characters (NPCs) continue to change as a game world develops, even when they are off-screen within the running simulation. In this context, large language models (LLMs) can help these NPCs reconsider goals and plans. However, continuous model inference is limited by hardware, latency, cost, and context size.

This research treats remote inference as a limited runtime resource and studies two related but separate decisions. Research question 1 (RQ1) compares AgentCentric and EventCentric admission policies for selecting which pressure cases receive strategic reasoning. Research question 2 (RQ2) compares Verbatim and source-linked Summary memory representations for admitted strategic decisions.

To study these questions, this research developed a persistent NPC runtime with three cognitive levels. Cognitive level 0 (L0) handles routine mechanical actions, cognitive level 1 (L1) handles bounded local choices, and cognitive level 2 (L2) handles selected strategic problems. At L2, the configured remote model receives only actor-visible information and returns a typed proposal. Host checks the proposal, Validator checks any concrete action, and Authority alone commits world-state changes. Evaluation therefore focuses on recorded task outcomes rather than narrative quality.

Across 30 unique matched pressure blocks that each carried a budget of $B = 4$, AgentCentric reached the correct grounded terminal 57 times out of 300 possible opportunities, a rate of 19.00%, whereas EventCentric reached it 60 times, a rate of 20.00%. The EventCentric minus AgentCentric difference of 1.00 percentage point had a 95% paired bootstrap interval from -4.00 to $+5.67$ percentage points. Within the specified 15-point practical threshold, neither of the two policies demonstrated a meaningful advantage. Verbatim and Summary each achieved grounded world-action success in 283 of 288 runs, while Summary reduced Provider input-token use by 66.33%.

A separate L1 evaluation of 120 paired observations found 114 acceptable local decisions and 119 acceptable remote decisions. The local model did not meet the declared non-inferiority requirement. Its use was estimated to avoid United States dollar (USD) 0.89 to USD 1.79 in remote Provider charges per 1000 bounded decisions.

A separate Cognitive level of detail (LOD) evaluation recorded 362013 completed L0 action receipts without model calls. The controlled dialogue suite achieved an expected-route match in 175 of 180 observations. However, 145 of 240 natural response opportunities failed the L1 dialogue contract, so the current dialogue router was not reliable for that workload.

LIST OF ABBREVIATIONS

AI	artificial intelligence
CPU	central processing unit
GUI	graphical user interface
IRM4MLS	Influence Reaction Model for Multi-Level Simulation
L0	cognitive level 0
L1	cognitive level 1
L2	cognitive level 2
LLM	large language model
LOD	level of detail
NPC	non-player character
P95	95th percentile
RQ1	research question 1
RQ2	research question 2
UI	user interface
USD	United States dollar

1 INTRODUCTION



THIS chapter introduces the research problem, states the two research questions, explains the role of the supplementary Cognitive level of detail (LOD) evaluation, and defines the contributions and scope.

1.1 BACKGROUND AND PROBLEM

Currently, there are many types of mechanical game behaviours that can still be solved using finite state machines, behaviour trees, and manual scripting. For instance, patrol routes, tool durability, etc., all have very clear rules. Therefore, using deterministic programming to create solutions for such problems is not only cost-effective but also easy to debug. However, when the pre-designed behavioural patterns cannot meet or resolve the requirements of the NPCs, the advantages of using large language models (LLMs) become apparent.

For example, a merchant has committed to providing supplies to a clinic. Due to a road block, the merchant's caravan was delayed and did not arrive at the clinic in time. Consequently, the clinic ran out of necessary medical supplies when a new patient came to seek treatment. Developers could attempt to write numerous nested branches to address this situation, but doing so would increase the complexity of the code exponentially and make it extremely difficult to maintain.

Persistent non-player characters (NPCs) make this branching issue worse. They continue to change with the simulation and world. Even when the player is absent, an NPC maintains its physical state, assets, and goals while adjusting plans and consuming resources. When the player returns, the NPC continues from the state shaped in the player's absence rather than repeating actions or inventing memories of past events.

Research conducted on Generative Agents demonstrated that memory, reflection, and planning can enable characters to remain active throughout the duration of the simulation (Park et al. 2023). Research conducted on Concordia allows for separation of an agent's attempted actions from the environment's adjudication of those actions. This enables characters to generate behaviours within a regulated environment (Vezhnevets et al. 2023). Both studies demonstrate that large language models can contribute to character cognition. Developing applications based upon these types of studies requires further examination of resource allocation. Remote large models have issues with call latency, network stability, and accumulation of input and output costs. Prior work on LLM cascades and routing also treats model selection as a trade-off between response quality and inference cost (Chen,

Zaharia, and Zou 2024; Ong et al. 2025). Calling a remote large model for every action would therefore be difficult to sustain in a town with many recurring actions.

This study adapts the LOD concept from computer graphics to divide NPC cognition into cognitive level 0 (L0), cognitive level 1 (L1), and cognitive level 2 (L2). L0 executes predefined mechanical actions. L1 can use a local model for bounded choices within the NPC's current goal. L2 uses the configured remote model for strategic problems that require replanning. This division asks when each level is needed instead of using the strongest model for every action. Because L2 capacity is limited, each admitted `DecisionNeed` occupies one logical L2 session.

1.2 RESEARCH GAP AND MOTIVATION

If ten NPCs experience problems simultaneously, and the system is capable of managing only four of those problems using the remote model, then the NPCs closest to the player may be unaffected by the damaged bridge, whereas the NPCs on the other side of the river may be unable to reach their respective stores. Examining each NPC separately provides an easier means of identifying problems that directly impact the individual NPC (such as hunger or failed plans), while examining world events first provides an easier means of determining which resources and Actors are affected by the damaged bridge. Although both views help locate affected Actors, they may miss different problems, so it is necessary to investigate their effects.

A second issue involves the relationship between the lifespan of an NPC and its accumulated memories and experiences. Most decisions made by an NPC only utilize a small percentage of its accumulated memories and experiences. While retaining all available records minimizes loss of detail, it consumes a large amount of context. Utilizing summaries enables retention of more history, but it may also result in removal of important information that impacts whether the proposed actions are legitimate.

A model may give a convincing response without changing anything in the game. Success is therefore measured by recorded game results, not natural-sounding text. Host checks the model's typed proposal. If it contains a concrete action, Validator checks current game rules, and only Authority commits the resulting state change.

1.3 RESEARCH QUESTIONS AND OBJECTIVES

This study examines two separate decisions in the L2 process. Before L2 dispatch, the system must decide which `Queued DecisionNeeds` receive the limited sessions. After admission, it builds a `MemoryPacket` from the same `DecisionMemoryCandidateSet`, using either `Verbatim` records or a source-linked `Summary`. These two decisions are evaluated separately.

research question 1 (RQ1): How do the `AgentCentric` and `EventCentric` admission policies influence processing results and operational costs associated with NPC strategic tasks when model invocation budgets are limited?

AgentCentric primarily focuses on pressure cases local to each NPC, such as failed actions, interrupted plans, and expiring commitments. EventCentric begins with Authority events or registered decision pressures and identifies affected Actors through analysis of dependencies between events, tasks, resources, and Actors. RQ1 compares how each policy chooses the pressure cases and how this choice affects grounded terminal success, unproductive L2 sessions and Provider use.

research question 2 (RQ2): For strategic tasks for which DecisionNeeds have been admitted into logical L2 sessions, can replacing whole candidate records packed by Verbatim with a source-linked Summary reduce input context while maintaining similar action quality? Does performance vary as more history accumulates?

Verbatim packs whole candidate records under the context ceiling, so lower-ranked records may not fit. Summary compresses the candidate set while retaining source links. RQ2 compares action quality and context use under both memory representations and examines whether Summary remains sufficient as an Actor accumulates more history.

The first objective is to compare AgentCentric and EventCentric under the same queued DecisionNeeds and logical L2 session budget. The comparison records admission choices, grounded terminal success, unproductive sessions, and Provider use.

The second objective is to compare Verbatim and Summary after admission to L2. Both conditions use the same candidate set and memory-context ceiling. This allows the study to compare task outcomes and context use without changing admission.

The model receives only the focal Actor's ActorCognitionView. Host checks its proposal, Validator checks any concrete action, and Authority alone commits world-state changes.

A separate supplementary objective is to examine whether L0, L1, and L2 take on distinct workloads in the implemented town. It measures L0 workload, the quality and cost of bounded local L1 decisions, and dialogue routing. It does not add a third research question.

1.4 CONTRIBUTIONS

Regarding system implementation, this paper describes a prototype of an ongoing functioning NPC town. Godot handles map rendering, player actions, and screen display. C# is used to save and update world state. The town includes 25 NPCs, all of whom utilize the same system for schedule management, physical states, assets, trade, relationships, commitments, dialogue, and memory management. The primary purpose of the Living Town Demo is to confirm that these functions can operate together within the same town. Interesting dialogues and unexpected behaviours observed during the Demo are recorded only as observations and are excluded from both experiments.

The study is based upon two controlled experiments. For RQ1, 30 different 10-pressure blocks are used with a fixed budget of $B = 4$. For each block, the queued needs and available L2 slots are held constant, but the admission policy is varied. The suite varies the degree of overlap in the top-four selections of the two policies, and does not repeat one fixed admission pattern. RQ2 uses the same candidate set and varies only the memory representation shown to the model. Both experiments utilize the same runtime paths for Provider requests, action execution and Authority commits. A scorer evaluates success from the terminal records produced by the system.

The supplementary Cognitive LOD evaluation provides separate system evidence. It compares local and remote handling of bounded L1 decisions, measures L0 and L1 workload across 60 game days, and evaluates dialogue routing in controlled and natural workloads.

1.5 SCOPE AND DELIMITATIONS

Each RQ compares only two conditions. RQ1 allows the same set of ten pressure cases to compete for four logical L2 sessions. EventCentric uses event dependencies to identify potentially affected Actors, but these dependencies are only used to select NPCs. They do not appear in the model prompts or determine the next action for the NPC. RQ2 starts after a `DecisionNeed` is admitted to L2. Both conditions start from the same candidate set and build the `MemoryPacket` as either `Verbatim` or `Summary`.

RQ1 is separate from RQ2: RQ1 compares the end-to-end effects of two admission policies using the same queued set and L2 budget, while RQ2 begins only after a `DecisionNeed` has been admitted. This separation prevents the RQ1 policy choice from affecting RQ2's memory representation comparison.

The 30-block design estimates the end-to-end effect of each admission policy. The selected cases vary across the 30 blocks; however, the policy-exclusive needs continue to go through the same model, planning contract, Validator, and Authority path. Therefore, differences between the policies include both the admission choice and the downstream resolvability of the selected tasks. Shared admissions allow some comparison under identical request inputs.

LITERATURE REVIEW

This chapter reviews the main ideas used in this study. It covers limited cognition, decision scheduling, long-term memory, and world adjudication. It ends by stating the research gap.

2.1 LIMITED LLM COGNITION AND INFERENCE BUDGETS

When discussing LOD, people usually think of reducing central processing unit (CPU) and memory overhead during rendering; however, LOD has also been applied to character behaviour. Niederberger and Gross (2005) studied the cognitive LOD of real-time characters. Wißner, Kistler, and André (2010) applied different levels of artificial intelligence (AI) processing to virtual characters in games and simulations. Both studies demonstrate that when computational resources are limited, the system does not need to ensure that each character always consumes the most expensive resources, but can adjust according to current needs.

Graphics LOD and cognitive LOD cannot be evaluated in the same way. Visually simplifying a graphic can create a visible error; however, reducing the behavioural processing of an agent means that agent will either miss completing a task or complete it late. Navarro, Flacher, and Corruble (2011) looked at both computational cost savings and the behavioural differences of agents in a very large urban simulation.

Using fewer model calls for LLM-based NPCs shows lower resource use, but does not show whether the affected NPC completed the task.

Sunshine-Hill (2013) pointed out that distance often does not reflect the importance of an object. This problem is even more obvious in a persistent NPC world. An NPC far away from the player may change due to fulfilling commitments, sharing resources, or having important information, so it needs more model attention. Therefore, this paper fixes the scene scope and number of NPCs, and does not compare nearby versus distant NPCs. RQ1 compares two admission policies for selecting the problem to process.

Model requests are not like ordinary game logic, which can be completed instantly every frame. They can be delayed by network latency, time-outs, and retries. Even with identical inputs, the model may return different answers. Therefore, this paper defines the budget in logical L2 sessions, while recording physical Provider attempts separately. Even if a session retry occurs, it only occupies one logical L2 session, but the time and cost incurred by the retry are included. Task success, latency, valid results, and request cost must be analysed separately.

LLM routing research studies a related trade-off between model quality and inference cost. FrugalGPT uses a model cascade, while RouteLLM learns when to select a weaker or stronger model (Chen, Zaharia, and Zou 2024; Ong et al. 2025). These studies support selective model use, but they route independent language tasks. They do not measure how much work remains at a deterministic NPC level or whether a proposed action reaches an Authority commit.

The reviewed cognitive LOD studies demonstrate the possibility of employing varying levels of processing, but they fail to indicate whether the local model utilized in the current study will be able to handle bounded decisions, and whether dialogue requests can be reliably routed among L0, L1, and L2 during runtime. These two issues are addressed in the supplementary Cognitive LOD evaluation, separate from RQ1 and RQ2.

2.2 NPC AND MULTI-AGENT DECISION SCHEDULING

Research on interactive narrative has already discussed the differences between character-centric and event-centric approaches. Poulakos et al. (2015) compared two ways of organizing the story world: one starting from what characters can do, and the other from the main events that occur in the world. Shoulson et al. (2013) planned by combining events with the required character positions, avoiding calculating all possible actions for each character separately. Flores and Thue (2017) determined which characters are related to an event based on social relationships, causal relationships, time, and place.

This paper borrows the terms *AgentCentric* and *EventCentric*, but their role in this project is much smaller. Neither policy creates the plot or determines what happens next in the world. They only prioritise *Queued DecisionNeeds* and select which NPCs to process first when the logical L2 session budget is limited. Once a *DecisionNeed* is admitted, both policies enter the same model-request and action-execution flow, and the model is unaware of which policy is active. Therefore, RQ1 compares two admission policies, not two complete narrative systems.

The influence-based abstraction proposed by Oliehoek, Witwicki, and Kaelbling (2012) allows for recording only the significant influences between different local systems, without requiring each agent to retain all external states. This paper adopts a similar approach, recording only the event dependencies needed for selecting NPCs. These dependencies are internal system information and do not represent what the NPCs themselves know. What an NPC sees still depends on its own experience and current visible state. If the dependencies used by the system to find affected Actors are directly written into the model prompts, the *EventCentric* policy will gain additional information, and experimental comparisons will lose fairness.

Both policies may miss relevant problems for different reasons. *EventCentric* may miss a problem when event dependencies are missing or outdated. *AgentCentric* may detect a problem only after an external change has affected the Actor’s state, observations, or commitments. Therefore, the two policies are compared using the same problem set and

L2 budget.

2.3 LONG-TERM MEMORY REPRESENTATION

Long-term memory faces another problem: after a character has experienced many things, which experiences should be used for the current decision? Generative Agents store the character’s observation records and then retrieve past experiences based on recency, importance, and relevance (Park et al. 2023). Earlier K-line theory suggests that recall is the evoking of a part of the past state by current cues, rather than rereading the entire history each time (Minsky 1980). The two studies differ in their implementation, but both demonstrate that the content stored by the system is not necessarily the same as what the character can find and use in the current decision.

HippoRAG uses knowledge graphs and Personalized PageRank to find relevant information scattered across different records (Gutiérrez et al. 2024). AriGraph stores event experiences and general knowledge in an updatable graph model and uses this information to formulate plans in a textual context (Anokhin et al. 2025). These studies demonstrate that long-term memory can be organized and retrieved according to relationships. However, the memory structures, task contexts, and success criteria they use differ from those in this paper, therefore, they cannot directly determine whether Verbatim or Summary is more suitable for this system.

RQ2 uses situations that are prone to errors in long-term memory, including old messages, conflicting statements, and promises made long ago. LoCoMo studies temporal and causal relationships in long conversations (Maharana et al. 2024). LongMemEval breaks down long-term memory into several types of questions for testing, such as retrieving past information, connecting different conversations, judging chronological order, updating old knowledge, and refusing to answer when information is insufficient (Wu et al. 2025). These studies primarily examine the textual answers given by the model. Persistent NPCs go a step further: they must choose a specific action based on their memory, and then the system checks whether this action conforms to the current world rules.

A larger context window does not guarantee that the model uses all included information equally. Liu et al. (2024) found that model performance can change when relevant information appears at different positions in a long context. This gives an additional reason to evaluate the decision result rather than assuming that loading more history is always better.

To explain where an NPC’s memories come from, the system maintains a source record for each experience. Authority events record what actually happened in the world, while ActorExperienceReference records how the NPC learned about it. They might have participated personally, witnessed it, or heard about it from others. Therefore, an NPC can remember rumors or continue to believe outdated information. This content belongs to the NPC’s own understanding and does not change the world state preserved by the system. With these source records, the scorer can check whether the Summary only uses

information the NPC actually encountered, and can also determine whether the NPC relied on later-outdated information when taking incorrect actions.

However, knowing where a message came from does not mean that the message is necessarily correct. The NPC may only have seen part of the event, and what they heard from others may be incomplete. The system records how the NPC obtained the message, not what the NPC believes as world fact. Therefore, RQ2 requires that the Summary retain the source and changes of the information. If the summary presents rumors as firsthand accounts by the NPCs, or continues to use old information that has been refuted by new events, it will be judged as incorrect, even if the text reads smoothly.

2.4 AUTHORITATIVE PERSISTENT-WORLD SIMULATION

A generative agent’s statement of its intention to do something does not necessarily mean that this action has actually occurred in the game world. Influence Reaction Model for Multi-Level Simulation (IRM4MLS) separates the agent’s proposed influence from the environment’s response (Morvan, Veremme, and Dupont 2011). Concordia also records what the agent tried, how the environment judged it, what actually happened, and what the agent saw (Vezhnevets et al. 2023). They all follow the principle that the subject proposes an action plan, and the environment determines whether the action can occur according to its own rules.

In this system, the model can only propose one typed decision and cannot directly modify the world. The Host first checks the request and the model’s returned proposal, and the Validator then determines whether the action conforms to the current world rules. Only after the action passes the check does the Authority commit the world-state change. Therefore, the model’s statement that it has completed a transaction or fulfilled a commitment does not mean that the corresponding item, currency, or commitment state has actually changed.

tau-bench also evaluates the tool agent based on the actual execution results. It checks the database state after a tool call and determines whether the execution process adheres to business rules (Yao et al. 2025). While it studies customer service tasks, this evaluation method is also applicable to persistent NPCs because both require distinguishing between what the model says and what the system actually executes. This paper uses the terminal result returned by the runtime as the basis for task completion. If execution fails, the system also records whether the problem occurred in the request format, action parameters, rule checks, or the Authority commit stage. Natural language is only used to express the Actor’s decisions, and the actual task outcome must be proven by a valid terminal record.

2.5 LITERATURE SYNTHESIS AND RESEARCH GAP

The literature reviewed above covers four areas that form the basis of this study. First, Cognitive LOD regulates processing effort according to need. Second, narrative and multi-

agent studies describe three methods for identifying relevant agents: character-centred, event-centred, and influence-based. Third, long-term memory research examines retrieval and compression across extended histories. Fourth, authoritative simulations distinguish between proposed actions and committed outcomes.

Cognitive LOD and LLM routing allocate resources at different levels. Cognitive LOD regulates behavioural processing according to an agent’s current importance, whereas FrugalGPT and RouteLLM select among models for individual language tasks (Niederberger and Gross 2005; Wißner, Kistler, and André 2010; Navarro, Flacher, and Corruble 2011; Chen, Zaharia, and Zou 2024; Ong et al. 2025). Although these approaches support selective inference, neither determines which unresolved NPC need should use a scarce L2 session or whether it ultimately reaches an executable endpoint. RQ1 therefore evaluates budgeted admission using Authority-grounded task success rather than model-use rate alone.

The scheduling literature uses various signals to identify relevant agents. Character-centred approaches start with capabilities, event-centred approaches with events and participants, and influence-based abstraction with dependencies between local systems (Poulakos et al. 2015; Shoulson et al. 2013; Flores and Thue 2017; Oliehoek, Witwicki, and Kaelbling 2012). These approaches integrate selection into narrative planning or decentralised reasoning. RQ1, however, treats AgentCentric and EventCentric as admission policies. Both rank the same queue and use a common downstream model and action path, while EventCentric alone uses dependencies as a system-side ranking signal.

Many long-term memory systems change storage, retrieval, and representation together. Generative Agents rank experiences, HippoRAG and AriGraph use graph-based retrieval, and LoCoMo and LongMemEval assess access across extended histories (Park et al. 2023; Gutiérrez et al. 2024; Anokhin et al. 2025; Maharana et al. 2024; Wu et al. 2025). RQ2 keeps candidate selection, order, and the downstream action path fixed while changing the MemoryPacket representation. This isolates the packet representation when comparing context use and executable task success within the tested cases.

Authoritative simulation provides a common basis for evaluating the two comparisons. IRM4MLS and Concordia distinguish between an agent’s proposal and the environment’s response, while tau-bench scores the state produced through tool execution (Morvan, Veremme, and Dupont 2011; Vezhnevets et al. 2023; Yao et al. 2025). Both RQs therefore use Host, Validator, and Authority terminal records rather than treating model statements as completed world actions.

Previous work supports selective cognition, relevance-based scheduling, retrieved memory, and state-grounded evaluation. This leaves two comparisons for this runtime: AgentCentric versus EventCentric under the same fixed L2 budget, and Verbatim versus source-linked Summary while the candidate set and executable task remain fixed. The supplementary evaluation records how much bounded work L0 and local L1 handle in the running town.

SYSTEM DESIGN AND IMPLEMENTATION

This chapter describes the runtime used by the Living Town Demo, the two core experiments, and the supplementary Cognitive LOD evaluation. These parts use the same model-call, validation, Authority, and research-recording boundaries.

3.1 SYSTEM ARCHITECTURE

The system is developed using Godot 4.7.2 .NET and C#. Its root runtime object is `LivingTownProductComposition`. It owns the components for simulation time and changing world state, while Authority is the only part that can commit state changes. The Godot scene is responsible for screen, player input, and interactive display, and it does not store a separate copy of NPC state. Even if an NPC is off-screen, out of the player's camera view, or not drawn by Godot's renderer, the NPC's physical state, item use, relationships, and daily routine continue to be updated. When an NPC comes back into view, Godot will show the same Actor state again.

Actor information is divided into world state and the Actor's own cognition. Location, physical state, inventory, and established commitments belong to world state. Goals, plans, beliefs, experiences, and subjective memories belong to the Actor itself. System generates an `ActorCognitionView` for focal Actor at specific time which only contains information the Actor can know at that moment. Hidden physical states, private states, inventories, scoring answers, and system dependencies used by `EventCentric` for other NPCs do not appear in model prompts.

Figure 3.1 summarizes the separation between presentation, actor-visible cognition, model inference, and authoritative state changes.

3.2 COGNITIVE LOD AND DECISION FLOW

3.2.1 AUTONOMY DECISION FLOW

All cognitive levels have the same persistent runtime and Authority boundaries. L0 does not invoke a model. L0 continues current plans and takes actions that are based on clearly defined rules. L1 uses the configured local Qwen3-8B model to choose from a bounded set of actor-visible options. L1 can only return `Choose`, `Defer`, or `RequestEscalation`. If the model requests an escalation, Host will check if there was sufficient reason for the escalation, and the escalation request cannot directly create a `DecisionNeed` or call L2.

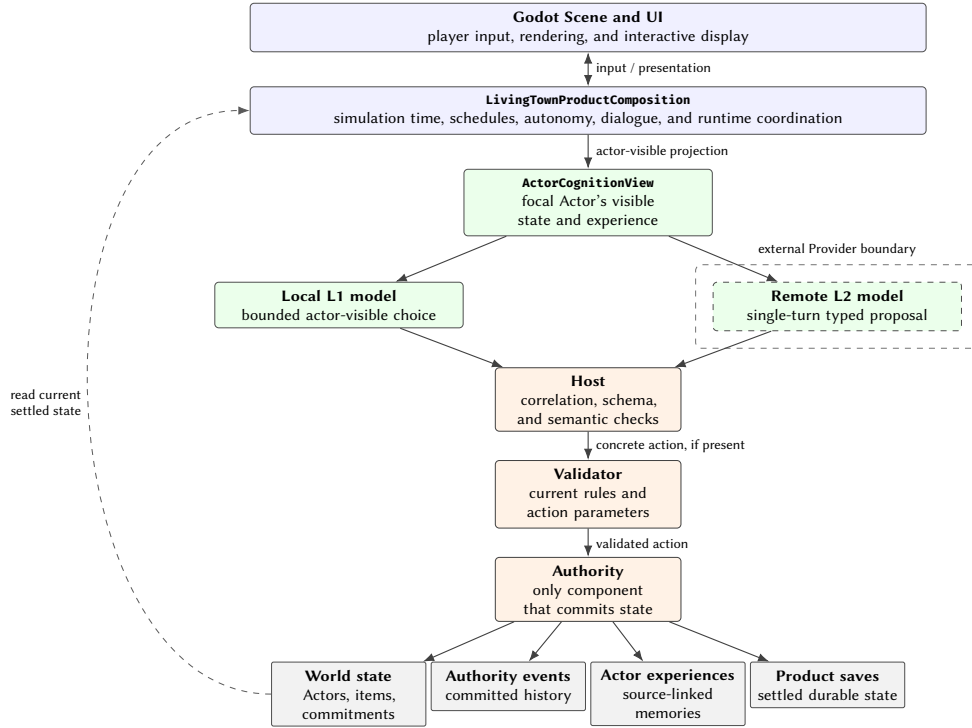


FIGURE 3.1: Runtime architecture and authoritative state boundaries.

Only after issues such as activation discovery, no feasible local action, goal or commitment conflicts, strategic plan failures, required strategic dialogue responses, or accepted L1 escalation requests pass deterministic Host checks does the system create a DecisionNeed. Normal processing moves it from Created to Queued, then InFlight and Resolved. Superseded can end a Queued or InFlight need, while Aborted can end any active state. A retryable transport failure returns InFlight to Queued.

L2 uses a single-turn L2AgentSession. The L2 planning contract defines this session, the L2 tool catalogue, and the proposal schemas. The configured remote model only returns a typed decision proposal. Host checks request correlation, permitted tool schema, and semantic references in the response, and then creates or modifies NpcPlan, verifies information, defers a decision, cancels a plan, or returns a strategic invitation response. The model does not directly control NPCs or execute plans frame-by-frame.

3.2.2 DIALOGUE ROUTING STRATEGY

When a recorded semantic turn generates a response opportunity for an NPC, dialogue routing begins. An invitation is checked by the deterministic L0 path first. Based on the NPC's visible relationship values, routine invitation preference, and current commitments, the Host determines if there is one clear ordinary response. A clear invitation can be accepted or declined without calling a model.

Instead of making a guess about what will happen next after receiving an invitation that requires a consequential decision, the Host may register the unresolved invitation

as a `DecisionNeed`. All additional response opportunities enter the local L1 route. The local model receives the actor-visible state of the responding NPC, the current activity of the responding NPC, any incoming text, and one visible evidence reference. This L1 dialogue contract provides two options for engaging in dialogue: (1) choose with a typed local reply, and (2) `request_escalation` with a reason and evidence. Generally speaking, greetings, ordinary information, simple advice, and low-risk emotional responses are typically handled at L1. When an L1 output is unavailable or invalid, it ends at the L1 failure stage and is not automatically sent to the remote model. L1 does not have the capability of creating a `DecisionNeed`, nor does it have the capability of calling L2 directly. Only when an escalation is requested using an allowed reason, citing the supplied actor-visible evidence, and describing a strategic signal, does the Host accept the escalation. Strategic signals include commitments, debts, threats, major relationship changes, medical or body deadlines, and repeated visible failures. Upon acceptance of an escalation request, a dialogue `DecisionNeed` is created.

Subsequently, L2 utilizes the same actor-visible memory preparation path as other town decisions and requests a single typed strategic reply from the remote model. Once a valid L1 or L2 reply has been received by the runtime, the runtime records the semantic reply, the surface text of the reply, the history event associated with the reply, and any permitted social appraisal. A completed L2 reply resolves its `DecisionNeed`. When there are failures with context preparation, Provider execution, or response validation, the pending response is abandoned and the failure stage is recorded. Model text alone cannot create a commitment or directly alter authoritative world state.

Figure 3.2 combines L0/L1/L2 routing with the grounded-action path followed after a `DecisionNeed` is created.

3.3 AUTHORITY AND VALIDATION

A `PlanStep` can have a concrete `GameActionSpec`; however, some `PlanSteps` describe only an objective, a target, and a desired result. The deterministic runtime attempts to find a legal `GameActionSpec` in the action catalogue that matches those fields. If there is no matching action, the plan may be structurally valid, but it cannot reach a grounded endpoint. Once a concrete action is available and submitted, `Validator` re-checks distance, access permissions, resources, target version, and action rules. After these checks pass, `Authority` commits the world-state change. A rejected action does not change the world.

This check must occur when an action is submitted, not just once before the model generates a plan. For example, two NPCs might simultaneously plan to buy the last item. Both plans might meet conditions at creation, but the item can only be bought by one person. After the first action is successfully committed, the inventory and target version have changed, and the second action is rejected.

Therefore an action failure is not necessarily due to the model remembering wrong information. It could also be because the world state has changed after the plan is generated.

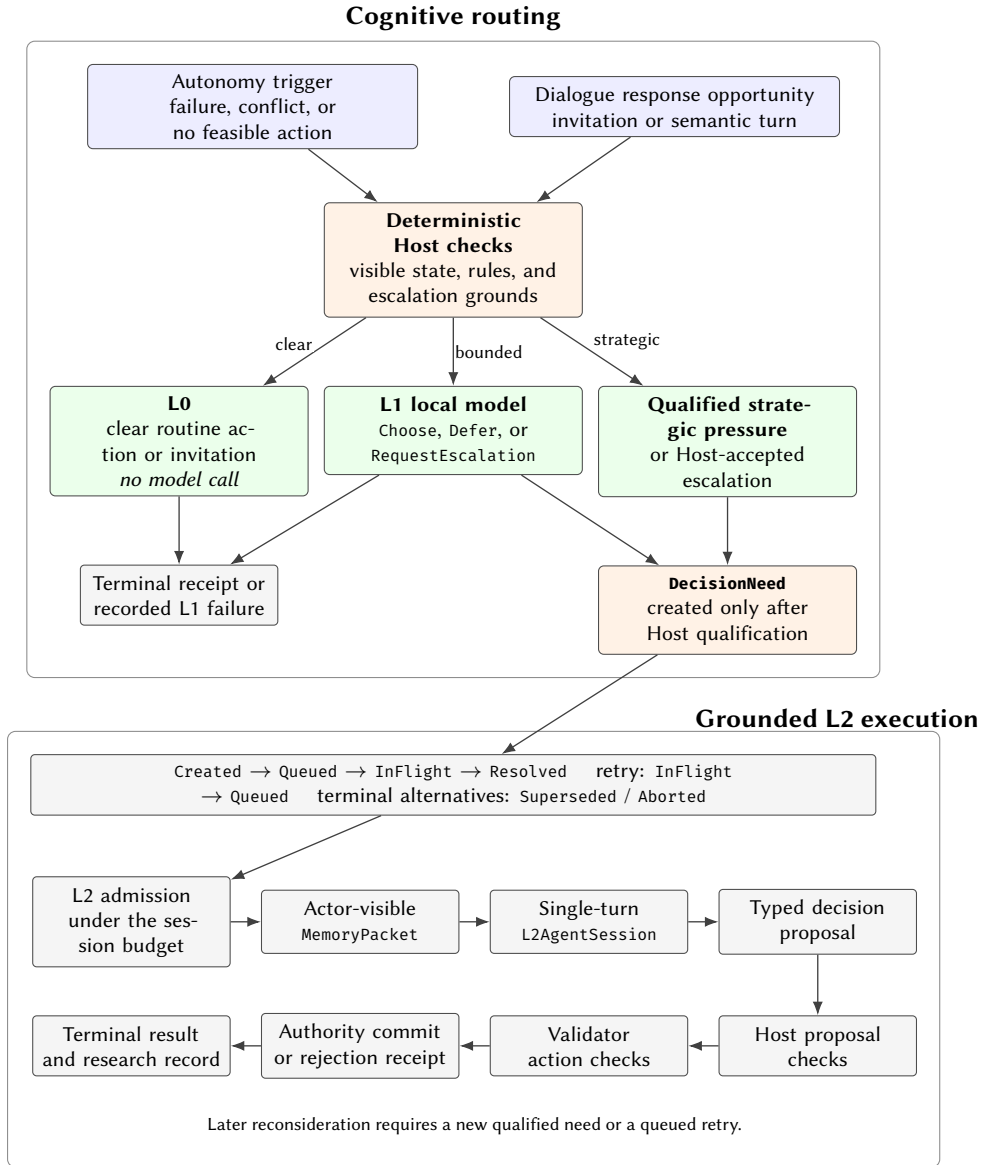


FIGURE 3.2: Cognitive LOD routing and grounded L2 execution.

The experiment records the state seen by the model, the action commit time, and Authority's final receipt to determine whether the problem occurred in the memory, planning, validation, or commit phase.

When an NPC learns of a failure, it can only view the part that the Actor has reason to know about why this failure occurred. The system may inform the NPC that "the transaction was not completed," or "the target is no longer usable." However, it cannot provide direct access to other Actors' actual inventory or internal version number, or expose all of the Validator's underlying reasons. This visible failure information may generate additional ActorExperienceReferences, and if the initial problem continues to persist, these generated references may be used to support subsequent decisions. A single instance of a failure does not permit continued use of the same L2 session for an unlimited period

of time. If an NPC requires reconsideration, the system must either create a new qualified `DecisionNeed` or leave the current one `Queued` for a later logical L2 session.

The catalogue of action types contains a set of commonly used typed actions. Logging, mining, fishing and gathering are all expressed as `RegionOperation`; the differences between these types are simply related to their respective requirements and objectives. Movement actions first bring Actors to an interactive location. Crafting, consumption, rest, transfer of assets, listing of items for exchange, service trading, change of location state and change of equipment each follow their own validation and commit paths. Typically when new items are added only the data is needed and no new free-form tools for models are necessary.

3.4 PERSISTENCE AND MEMORY

World events are stored separately from the Actor's own experience. Authority events record what actually happened. Actors involved in the event, as well as those meeting observation criteria, receive a source record pointing to the event. If the message is later relayed by someone else, the recipient receives a new experience retaining the relationship between the narrator and the original event.

Witnessing an event requires Actors to be in the same spatial layer and meet the location's access rules. Normal observation distance is two meters. Memories from different Actors can contradict each other, but each record entering the candidate set must meet three conditions: it belongs to focal Actor, recording time is no later than `CueSimTime`, and a valid information source can be found.

Through the debug interface, researchers can see Actor states, memories and action receipts, but all of these items do not get copied verbatim to model prompts. Product saves also have explicit write times. A product can only be saved after the current settlement has ended and there are no Provider requests being processed. Product saves use readable unversioned JSON files. These files contain information about the settled tick, the durable states of both players and NPCs, gameplay and social state, history events, conversations, the dialogue surface, the active RQ1 admission policy and RQ2 memory condition, and autonomy state. The user interface (UI), cache, navigation displays, access credentials, and hidden model states are not included in the save file. Upon loading, the above-mentioned elements are rebuilt. Experiment restoration is done using fixtures and is therefore separate from product saves.

3.5 LIVING TOWN DEMO

The Living Town Demo uses the same `LivingTownProductComposition`, NPC state, schedules, action rules, dialogue runtime, memory, Validator, and Authority paths as those used for supplementary experiments.

Figure 3.3 shows L0–L1–L2 routing in the running Demo.

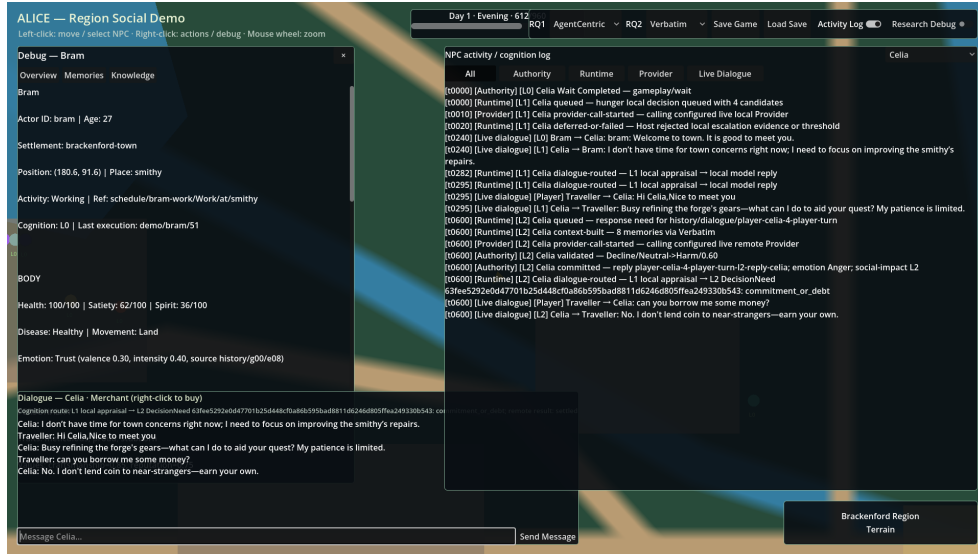


FIGURE 3.3: L0–L1–L2 cognition routing in the Living Town Demo.

Each NPC starts with a configured residence, profession, schedule, inventory, physical condition, and personal history. Inventory, physical condition, relationships, commitments, and plans can change during simulation. This content is data-configured so the system does not need to write a separate NPC subclass for each profession. Town runtime also handles resources, agriculture, food and satiety, tool durability, economy, disease and treatment, work, desires, debts, and commitments.

Actor relationships are directional. For example *A*'s trust in *B* does not necessarily equal *B*'s trust in *A*. System records relationships based on six dimensions: familiarity, trust, affection, respect, fear, and grievance. Dialogue can lead to invitations, friendships, partnerships, or marriages, but these changes must be verified through identity checks and Authority commits. They cannot be announced solely by model in dialogue.

RESEARCH METHODOLOGY

4

This chapter explains how the two core experiments were run. The experiments share one runtime, but they test different parts of the decision process. A separate supplementary evaluation examines how L0, L1, and L2 divide work in the implemented system.

4.1 OVERALL EXPERIMENTAL DESIGN

The matched-pairs experimental design is used for RQ1 and RQ2. Both conditions in a matched pair begin at the same initial state. The world state, `DecisionNeed` queue, cache, dialogue records and Provider state generated from the first condition are not passed to the second. The 30 pairs of RQ1 were distributed across 30 fixed workers. Each pair ran both conditions sequentially on the same worker using counterbalanced order, and each condition's four Provider calls were issued sequentially. RQ2 remained serial.

Both conditions within a matched pair start from the same frozen world snapshot and share the same model profile, Validator, Authority, action definitions, and treatment-neutral prompt rules. The actual model inputs are not identical. In RQ1, the two admission policies may select different focal Actors. In RQ2, the two memory conditions intentionally use different representations. Internal system information used to differentiate between conditions is not sent to the model.

4.2 RQ1 DESIGN

RQ1 simulates a straightforward scenario: ten eligible, `Queued DecisionNeeds` await processing simultaneously but the system can only admit four logical L2 sessions to a remote model. There are 25 persistent NPCs in town with a logical L2 session budget of four, denoted $B = 4$. `DecisionNeeds` that don't receive a slot are denoted as `missed_due_to_budget`, and the system does not issue a Provider request for them. Sessions that have received a slot can be retried according to uniform rules if a network failure occurs. Retries increase waiting time, number of physical Provider attempts, and cost but still belong to original session and do not occupy fifth slot.

Each RQ1 block includes 10 actor-distinct pressure cases across 5 domains: care and household, obligations and logistics, tools and maintenance, food and production, and trade and harvest. Within the hidden test-case ledger there exist 5 expected commit families and 3 expected defer families. Across the complete suite, each treatment includes 60 cases

requiring an Authority commit and 60 cases where a justified defer is the expected terminal. These expected outcomes are used solely by the scorer and are not part of the model input.

Both admission policies begin with the same decision-pressure snapshot but use independent queues and world states. The system processes issues requiring immediate responses first, then compares deadlines. If an issue waits too long, the system calculates a deadline based on the fixed `StarvationAge`. `AgentCentric` and `EventCentric` rankings only affect order when these common rules are tied. Remaining ties are sorted by `CreatedAt` and `NeedId`. This ensures stable replay of the same input while applying the basic fairness rules equally to both policies.

`AgentCentric` starts with local issues currently encountered by an NPC, such as action failure, duty expiration, or plan failure. `EventCentric` starts with Authority events or registered decision pressures tracing affected Actors along typed dependency edges. System first identifies intermediate objects such as tasks, commitments, locations, resources, capabilities, information, or organizations from events and then finds relevant Actors. These dependencies are mapped to three rank bands: E0, E1, and E2.

Lookup does not require the model to read text and guess event dependencies. Only one-hop lookup is performed at runtime. If new issues arise during the current round of checks, these issues are handled in the next window, preventing an event from continuously spreading in the same round and repeatedly creating `DecisionNeeds`.

The experiment utilizes 30 distinct public town-context blocks. There is one matched pair for each block and no blocks are repeated. Each block includes ten simultaneous `DecisionNeeds` that compete for four logical L2 sessions. The overlap between the `AgentCentric` and `EventCentric` top-four selections is fixed at zero, one, or two shared admissions, with ten blocks in each overlap tier. `AgentCentric` and `EventCentric` both make four logical Provider requests per block. In total, 240 logical requests were made across all 30 pairs.

4.3 RQ2 DESIGN

RQ2 starts after a `DecisionNeed` has been admitted to L2. Each matched pair shares the focal Actor, need, and `CueSimTime`. Both memory conditions use the same actor-visible state and ranked candidate set. The system ranks candidate records by relevance, recency, and importance, using a weight of 1 for each factor, and uses the same token-counting method. Verbatim and Summary therefore start from the same records and order but construct different `MemoryPackets`.

Verbatim packs whole candidate records in ranked order under a memory-context ceiling of 8192 estimated tokens. If the next record does not fit, packing stops. It does not truncate that record or skip it for a shorter one. The packed records retain their time, Actor, action, and source, but lower-ranked records may be absent from the `MemoryPacket`.

Summary uses the same complete candidate set but compresses its content. The experiment uses 36 candidate sets, each with a source-linked Summary artifact generated by the same model and prompt. The Summary generator can read supersession relationships

between source records. Each Summary statement must point to at least one source ID. Information that is outdated or replaced by later events must also be marked. The Summary condition reads these artifacts and does not generate new ones during decision requests.

RQ2 arranges test materials based on both problem stratum and history load. Six problem strata are as follows:

TABLE 4.1: RQ2 problem strata.

Category	Problem Type	Description
P1	Simple Current State	Requires information that is still valid
P2	Stale State	Requires identifying information that has become invalid or been replaced
P3	Conflicting Reports	Different records give different accounts of same event
P4	Commitment Lifecycle	Requires tracking commitments with long durations
P5	Failed-Plan Revision	After original plan failed adjustments need to be made based on new situation
P6	Salient Distraction	History contains conspicuous information that is irrelevant to task

Each problem stratum is further divided into six history-load tiers:

TABLE 4.2: RQ2 history-load tiers.

Tier	Length of Complete Candidate Set
T1	≈ 4000 to 6000 tokens
T2	≈ 6000 to 9000 tokens
T3	≈ 9000 to 12000 tokens
T4	≈ 12000 to 18000 tokens
T5	≈ 18000 to 23000 tokens
T6	≈ 23000 to 30000 tokens

Token counts in the table refer to the complete candidate set. Both treatments select or summarize the information they need from this set.

Six problem strata are tested separately at six different history-load tiers. This results in total of $6 \times 6 = 36$ stratum-tier cells.

Each stratum-tier cell is run independently eight times using the same material. Each time Verbat im and Summary receive one logical Provider request, and these two requests

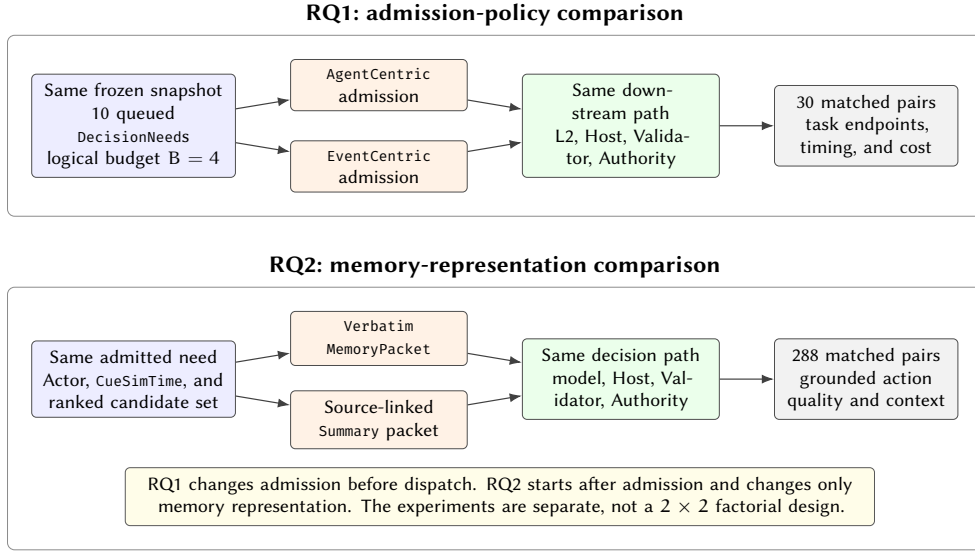


FIGURE 4.1: RQ1 and RQ2 experimental separation.

form a matched pair. Therefore the experiment consisted of $36 \times 8 = 288$ matched pairs and $288 \times 2 = 576$ logical Provider requests.

Figure 4.1 shows where the two comparisons intervene in the shared runtime. RQ1 changes admission before dispatch, whereas RQ2 starts after admission and changes only the memory representation.

4.4 MEASURES AND OUTCOME DEFINITIONS

RQ1 has a primary outcome measure of `task_success_rate`, which was computed by counting the number of correct grounded terminals among the 10 opportunities in each treatment and block. A correct terminal is defined as either the expected Authority commit or an explicitly expected justified defer. The pressure cases that do not receive one of the four available slots will not be able to reach a successful terminal within the admission window. The secondary measure of `unproductive_session_rate` represents the proportion of admitted L2 sessions that did not reach the correct grounded terminal. Diagnostic data includes Actor coverage, waiting time, starvation promotion, protocol failures, latency, retries and token use.

Cost and fairness are not combined into single total score. Cost records include input and output tokens, physical Provider attempts, wait time, Provider latency, cost, admission sorting time, and `tokens_per_authority_success`. If no Authority succeeds for a condition, the last item is marked as undefined. Fairness records include how many different Actors received services, how long they waited before first obtaining slot, how many Actors never obtained slot, and whether starvation promotion actually occurred.

Each issue also has a complete state record maintained from discovery through termination. Issues which do not obtain a slot, encounter a protocol error after admission, generate

a plan which is rejected by the Validator, or fail to generate a plan are recorded separately. These records show the stage at which each unsuccessful opportunity ended.

The primary result is `grounded_world_action_success_rate`. The model must make an action verification or deferral decision conforming to hidden rules. If a case requires an action, it must also obtain the expected Authority receipt. For comparison success difference between Summary and Verbatim in each matched pair is calculated first, and then two-sided interval and one-sided 95% lower bound are given using percentile bootstrap of 100000 paired resamples. In cases where the two memory conditions have inconsistent success directions, exact McNemar test is used for aggregation.

Other records explain where failures occurred including `MemoryPacket` estimated token issues, whether necessary sources were retained, whether source links were correct, protocol errors, action and parameter mismatches, Validator rejection, Authority failures, delays, and costs. Results are based on 288 matched pairs reporting both raw number and proportion of successes and failures. Breakdown of six problem strata and six load tiers is only used to observe which situations are more likely to show differences and is not considered primary conclusion.

4.5 STATISTICAL ANALYSIS

Each distinct town-context block represents one RQ1 matched pair. The 10 pressure cases in each block share four slots; therefore, the 300 pressure opportunities cannot be considered independent observations. The initial analysis determines the EventCentric minus AgentCentric difference for each block and subsequently summarises the 30 block-level differences. Reports include raw counts, the mean paired difference, a 95% interval based on 100000 paired percentile bootstrap resamples, and a two-sided exact sign test which excludes tied blocks. Descriptive analyses of the admission-overlap and failure-stage breakdowns are also conducted using the same sealed observations.

RQ2 uses a one-sided non-inferiority assessment as a method of statistical testing to determine if using a simpler alternative avoids a meaningful quality loss, not to show that the alternative has improved quality. Only when the one-sided lower bound of the interval is greater than -5 percentage points can it be said that Summary has essentially maintained Verbatim's action quality on these tasks. Percentage reduction in context is reported separately with target of at least 40% reduction.

Non-inferiority cannot be judged solely by whether two success rates are close. If the average success rate of Summary is similar to that of Verbatim but the one-sided lower bound is at or below -5 percentage points, the evidence is insufficient. Achieving 40% context reduction target only indicates shorter input and does not compensate for insufficient operational quality. This paper reports results for quality and context separately and then explains reasons in conjunction with information source coverage and failure stages.

4.6 DATA COLLECTION AND RUN INTEGRITY

The experiments use English prompts and the DeepSeek v4 Pro high-thinking profile. Each logical Provider request has a maximum output-token count of 16384 and a timeout of 300 seconds per attempt. The system only retries requests under three conditions: a network transmission failure, the parsing envelope from the Provider failed, or the output generated reaches the maximum allowable length. Retries use the same request, so no previous attempt content is included. Each request makes a maximum of three attempts with 5 second and 15 second delays prior to each retry. Once response has been successfully returned and parsed, the system does not re-run a request due to low-quality answers.

Each call record contains its request ID, model profile, model response, Provider-reported usage, response time, retry status, and final result. Costs associated with calling Provider are calculated from the reported token use and the pricing applicable on the date of data collection. When a failure has no Provider-reported usage, its cost is recorded as unknown rather than zero.

Success criteria for each case are stored in hidden scoring materials. Generally, the focal task involves performing an expected action within a designated time frame and receiving the associated Authority receipt. Some cases intentionally present conflicting or incomplete information. For these cases, verifying situation first or temporarily refraining from action may represent appropriate response. Acceptable alternatives are also stored in scoring materials. Following execution of both conditions in a matched pair, the scorer reads the endpoint record stored by the system. The scorer does not evaluate whether the model's response is written naturally, whether sufficient detail is provided regarding the model's thought processes, or whether a researcher subsequently determines the response is reasonable. Rather, evaluations are predicated solely upon what decisions were returned by the model and whether those decisions ultimately resulted in achieving the expected endpoint.

RQ1 contains 240 logical Provider requests and RQ2 contains 576, totalling 816. Coverage checking reads the declared matched-pair list and confirms that every expected result is present, rather than relying on the number of files in an output directory. Network failures follow the retry rules above, while completed requests with suboptimal responses are not replaced.

4.7 SUPPLEMENTARY EVALUATION OF COGNITIVE LOD

The supplementary evaluation examines how much work is handled at L0, L1, and L2 in the implemented runtime. It also tests whether the local model has practical value for bounded decisions. This evaluation does not create RQ3 and does not change the treatments or results of RQ1 and RQ2. Its observations are reported separately from the 816 logical Provider requests used by the two core research questions.

4.7.1 LOCAL L1 MODEL QUALITY AND COST COMPARISON

The L1 comparison uses 24 cases with predeclared acceptable outcomes. Eight cases require a direct local choice, four require a defer decision, and twelve require a request for strategic escalation. Each case is repeated five times. The local Qwen3-8B model and remote DeepSeek model receive the same actor-visible facts, candidate actions, decision instructions, and typed output contract. This produces 120 paired observations and 240 decoded model requests.

TABLE 4.3: Local-remote L1 comparison cases.

Expected L1 Route	Distinct Cases	Repeats per Case	Paired Observations
Choose	8	5	40
Defer	4	5	20
RequestEscalation	12	5	60
Total	24	5	120

The primary quality measure checks whether each decision matches a predeclared acceptable outcome. Secondary measures record route agreement, exact candidate or escalation-reason agreement, input and output tokens, reported reasoning tokens, latency, and failure location. A paired local-minus-remote quality difference is interpreted against a predeclared non-inferiority margin of -5 percentage points. The cost analysis treats the remote shadow request as one remote Provider request avoided by a deployed local L1 decision.

This comparison gives the same bounded L1 problem to two models. It does not compare L1 with a full L2 planning session, and neither branch commits world state.

Model order alternates within each pair, and all calls run in sequence. Repeats observe model variation but do not create new case types.

The 100000-sample bootstrap therefore resamples the 24 cases rather than treating the 120 observations as independent cases.

4.7.2 SIXTY-DAY COGNITIVE LOD WORKLOAD

The workload harness runs the current 25-NPC town for 60 game days without the graphical user interface (GUI). It reuses `LivingTownProductComposition`, the product schedules, body state, resources, action rules, dialogue runtime, and Authority. It is not a separate simplified simulation. The continuous trajectory produces three summaries: days 1-30, days 31-60, and days 1-60.

The run records L0 action receipts, completed and rejected L0 actions, autonomy L1 requests, L1 choices, defers, escalation requests and failures, L1 requests per NPC per ten minutes of configured Demo time, and autonomy L2 items recorded by the system. Per-NPC distributions are retained so that a town mean cannot hide concentration in a

small number of Actors. The two non-overlapping 30-day windows are also compared to show whether the workload changes as state and commitments accumulate.

Autonomy L1 requests use a deterministic settlement driver during this workload measurement. This prevents the descriptive run from making thousands of undeclared local-model calls. It also keeps the trajectory repeatable. The 60-day counts therefore measure how often the product produces L1 work, not how the world would change after thousands of live local-model decisions. Token use and avoided remote charges are estimated from the separate 120-pair L1 comparison and are not reported as directly observed 60-day Provider usage.

4.7.3 DIALOGUE TIERING EVALUATION

Dialogue is evaluated through a natural workload and a controlled routing suite. The natural workload shares the same 60-day town trajectory. It records L0 invitation settlement, LocalReply outcomes, L1 escalation requests, Host acceptance or rejection, actual L2 dispatch, terminal settlement, Provider tokens, latency, and failure stage. Host acceptance, L2 budget admission, Provider dispatch, and terminal settlement remain separate events.

Autonomy L2 items are recorded; however, they are not dispatched. Live Provider requests along the trajectory are used for dialogue routing. The two 30-day windows belong to one continuous world trajectory and are not independent experimental groups.

The controlled suite contains 36 actor-visible cases. Each case is repeated five times, producing 180 routing observations.

TABLE 4.4: Controlled dialogue routing cases.

Expected Route	Distinct Cases	Observations	Required Coverage
L0	12	60	Six clear invitation acceptances and six clear invitation declines
L1	12	60	Greetings, small talk, simple information, advice, and low-risk emotional replies
L2	12	60	Commitments, debt, threats, relationship change, medical deadlines, and repeated failure

L0 observations use deterministic invitation handling and do not call a model. L1 observations call the local model. L2 observations first call the local model to request escalation; the Host then decides whether the request can create a DecisionNeed, after which the remote model can produce a StrategicReply. The scorer records expected-route match, acceptable terminal outcome, false escalation of ordinary dialogue, missed

escalation of strategic dialogue, Host acceptance, actual L2 dispatch and completion, token use, latency, and failure stage.

The current town scheduler naturally creates four `CasualComment` conversations per game day and does not create NPC invitations. The natural workload therefore contains no L0 invitation observations. This is reported as the workload of the current product configuration. The controlled suite checks L0 invitation capability without changing NPC personalities or inserting new natural events. Player-input parsing is outside this Cognitive LOD evaluation.

5 RESULTS

This chapter reports the experimental results. It first checks the data and then reports RQ1 and RQ2 separately. The local L1 comparison, 60-day Cognitive LOD workload, natural dialogue workload, and controlled dialogue routing suite are reported as supplementary system evidence.

5.1 DATA INTEGRITY AND SAMPLE COMPLETION

The RQ1 dataset contains all 30 declared pairs, and the RQ2 dataset contains all 288 declared pairs.

TABLE 5.1: Core dataset completion and integrity.

Checking Item	Result
RQ1 Pairs	30 of 30
RQ2 Pairs	288 of 288
Invalid Pairs	0
Logical Provider Requests	816
Physical Provider Attempts	822
Retries after RQ1 Timeout	2
Retries after RQ2 InvalidResponseEnvelope	4
Coverage Check	RQ1 30/30; RQ2 288/288; 0 Blockers
Item Build	0 Warnings, 0 Errors
Protocol, Scoring, and Pair Run Checks	All Passed

The supplementary datasets contain 120 paired L1 observations, 180 controlled routing observations, and 240 natural response opportunities. The 60-day workload provides separate summaries for days 1-30, days 31-60, and days 1-60. These datasets use different observation units and are not included in the 318 core matched pairs, 816 logical Provider requests, or 822 physical Provider attempts shown above.

Each pair has corresponding model response, Provider usage, scoring results, and action endpoint records. Both processing conditions also went through the same Host, Validator, and Authority process.

The intervals shown here were calculated from matched-pair differences using 100000 paired percentile bootstrap resamples. For RQ1, a two-sided exact sign test was used for

block-level direction, and for RQ2, an exact McNemar test was used. The admission-overlap tiers and terminal failure stages have been reported to explain the overall RQ1 result.

5.2 RQ1 RESULTS

All differences in this section are calculated using EventCentric minus AgentCentric. Therefore, a positive task-success difference means that EventCentric was better than AgentCentric at task success. A negative unproductive-session difference indicates that there were fewer unsuccessful admitted sessions when EventCentric was used versus AgentCentric.

TABLE 5.2: RQ1 outcomes and fairness diagnostics.

Metric	AgentCentric	EventCentric	Difference between the two	95% Paired Interval
task_success_rate	57 of 300, 19.00%	60 of 300, 20.00%	+1.00 pp	[−4.00, +5.67] pp
unproductive_session_rate	63 of 120, 52.50%	60 of 120, 50.00%	−2.50 pp	[−14.17, +10.00] pp
Number of different Actors receiving services per pair	4	4	0	No change
Percentage of eligible pressure Actors who never received a slot	60.00%	60.00%	0 pp	No change
Wait among admitted Actors before first slot acquisition	95th percentile (P95) = 1 tick	95th percentile (P95) = 1 tick	0	No change
Starvation promotion	0 times	0 times	0	No change

These Actor-level diagnostics examine the 10 Actors representing the 10 pressure cases, not all 25 NPCs in the town, within a single admission window.

EventCentric reached 60 correct terminals, and AgentCentric had 57. The difference between EventCentric and AgentCentric was 1.00 percentage point, with a 95% paired interval from −4.00 to +5.67 percentage points. AgentCentric performed better in eight of the blocks; EventCentric performed better in thirteen blocks; and there were nine ties. An exact two-sided sign test yielded $p = 0.38331$. The point estimate and interval remained within the specified 15-point practical threshold, so neither policy demonstrated a meaningful advantage in the tested blocks.

The unproductive-session rates were 52.50% for AgentCentric and 50.00% for EventCentric. There was a difference of −2.50 percentage points between these two conditions, with an interval from −14.17 to +10.00 percentage points. This measure refers to the same

terminal chain from the perspective of unsuccessful admitted sessions. Therefore, it should be interpreted as a measure of session yield rather than as a separate outcome.

5.2.1 ADMISSION OVERLAP AND BLOCK-LEVEL RESULTS

TABLE 5.3: RQ1 results by top-four admission-overlap tier.

Shared Admissions	Blocks	Agent	Event	Difference	Agent Wins	Ties	Event Wins
0	10	21 of 100	20 of 100	−1.00 pp	3	3	4
1	10	18 of 100	20 of 100	+2.00 pp	2	4	4
2	10	18 of 100	20 of 100	+2.00 pp	3	2	5

Each policy made 120 admissions across the 30 blocks. Thirty admissions were shared by both policies, while 90 admissions were exclusive to each policy. As such, 75% of the admitted cases differed between AgentCentric and EventCentric.

The 30 shared admissions had the same request-body hashes in each condition. In each condition, thirteen of those cases were successful. There were nine shared admissions that were successful under both conditions, thirteen failed under both conditions, four were successful only under AgentCentric and four were successful only under EventCentric. The symmetric discordance shows that identical admitted tasks could still receive different model outcomes. Of the policy-exclusive admissions, AgentCentric succeeded in forty-four of ninety cases and EventCentric succeeded in forty-seven of ninety cases. The overall difference of three successful terminals came from cases that were selected by only one policy. Differences across all three overlap tiers were small and did not show a clear progression with increasing overlap.

5.2.2 FAILURE-STAGE DECOMPOSITION

TABLE 5.4: RQ1 terminal outcomes among admitted L2 sessions.

Observed Outcome	AgentCentric	EventCentric	Total
Correct AuthorityCommitted terminal	40	37	77
Correct JustifiedDefer terminal	17	23	40
Host accepted CreatePlan; no grounded endpoint	40	34	74
UnknownItem	15	12	27
InvalidProposal	5	11	16
UnknownTarget	2	3	5
ResponseFailure	1	0	1
Total	120	120	240

All 123 unsuccessful admitted sessions ended with an outer ValidatorRejected terminal. The nested Host records show the failure causes depicted in the failure-stage table. In

terms of success rates, cases expecting an Authority commit were successful in seventy-seven of 120 admissions, a rate of 64.17%, and cases expecting a justified defer were successful in forty of 120, a rate of 33.33%. Eighty of the cases expecting a justified defer were unsuccessful. Of these eighty unsuccessful expected-defer cases, seventy-four produced a structurally accepted `CreatePlan` but had not reached the required grounded endpoint. This was the most common failure under both policies and indicates that the current plan and defer path limited the yield of admitted sessions regardless of the ranking policy used.

5.2.3 PROVIDER USAGE AND TERMINAL OUTCOMES

TABLE 5.5: RQ1 Provider usage and terminal outcomes.

Diagnostic Item	AgentCentric	EventCentric
Logical Provider Requests	120	120
Physical Provider Attempts	120	122
Timeout Attempts	0	2
Input tokens	506913	503272
Output tokens	216830	268770
Total tokens	723743	772042
Median successful-response latency	25.24 s	29.24 s
AuthorityCommitted	40	37
JustifiedDefer	17	23
ValidatorRejected	63	60

Two EventCentric attempts timed out and were retried using the same frozen requests. Both retries completed, so all 240 logical requests produced usable responses. No request reached the output-token limit.

In contrast to AgentCentric, EventCentric consumed 0.72% fewer input tokens, 23.96% more output tokens, and 6.67% more total tokens. Additionally, EventCentric had a higher median successful-response latency than AgentCentric. The above data reflects Provider use after admission. Since the two policies were not selecting identical task sets, these figures do not isolate policy execution cost.

5.3 RQ2 RESULTS

Verbatim and Summary each achieved grounded world-action success in 283 of 288 runs, or 98.26%. Their action results were almost identical, but Summary substantially reduced the amount of historical data sent to the model.

Of the 288 pairs, 278 succeeded under both conditions. Five had only Verbatim successful, and five had only Summary successful. No pair failed under both conditions. The

TABLE 5.6: RQ2 quality and context reduction.

Metric	Verbatim	Summary	Summary – Verbatim	95% Paired Interval
grounded_ world_action_ success_rate	283 of 288, 98.26%	283 of 288, 98.26%	0.00 pp	[−2.08, +2.08] pp
Average MemoryPacket token	7461.56	111.17	Decrease 98.48%	[98.37%, 98.58%]
MemoryPacket token total	2148928	32016	Decrease 98.51%	Target is at least 40%
Required- source retention	288 of 288	288 of 288	No difference	All retained
Source-link fidelity	Not applicable	288 of 288 Pass	Not applicable	All passed

exact McNemar test yielded $p = 1.0$, indicating that failures did not significantly favour either memory representation.

The success rate difference between Summary and Verbatim was 0. The one-sided 95% bootstrap lower bound was -1.74 percentage points, higher than the -5 percentage-point non-inferiority margin. Therefore, RQ2 met the non-inferiority criterion. In other words, in these tasks, Summary did not show a drop in success rate beyond the acceptable margin due to memory compression.

Candidate records and necessary sources were valid in all 288 pairs, and all source links for Summary passed the checks. The ten discordant pairs were therefore not caused by the system missing memories, truncating necessary sources, or missing sources in the Summary.

5.3.1 RESULTS UNDER DIFFERENT HISTORY LOADS

No continuous trend of "the longer the history, the worse the Summary performance" was observed across the six history-load tiers. The Summary performance was 4.17 percentage points lower in T1 and T6, 4.17 percentage points higher in T2 and T5, and the same as Verbatim in T3 and T4.

TABLE 5.7: RQ2 outcomes by history-load tier.

History-Load Tier	Verbatim Success	Summary Success	Summary – Verbatim	MemoryPacket Reduction
T1	48 of 48	46 of 48	−4.17 pp	97.91%
T2	46 of 48	48 of 48	+4.17 pp	98.99%
T3	48 of 48	48 of 48	0 pp	98.43%
T4	48 of 48	48 of 48	0 pp	98.57%
T5	46 of 48	48 of 48	+4.17 pp	98.67%
T6	47 of 48	45 of 48	−4.17 pp	98.29%

Each tier has 48 matched pairs, and very few pairs actually produce different results. Therefore, this table can help identify situations requiring repeated testing, but the overall judgment should still be based on all 288 matched pairs.

The six problem types did not show a fixed direction. Summary was 6.25 percentage points higher in Simple Current State and 2.08 percentage points higher in Conflicting Reports. It was approximately 2.08 percentage points lower in Commitment Lifecycle, Failed-Plan Revision, Salient Distraction, and Stale State. These differences were caused by only one to three sets of discordant results and cannot be used to determine which type of problem is more difficult or more suitable for using Summary.

5.3.2 CONTEXT AND PROVIDER USAGE

The estimated token count for MemoryPacket was calculated using `utf8_bytes_div4_v1`. Verbatim required an average of 7461.56 estimated tokens, while Summary required an average of only 111.17 estimated tokens, a reduction of 98.48% on average. Based on the total estimated token count for all matched pairs, the reduction was 98.51%, substantially exceeding the 40% target.

MemoryPacket only reflects the memory portion. To confirm whether this reduction also occurred in actual Provider requests, the experiment also tallied the token usage returned by the Provider.

TABLE 5.8: RQ2 Provider usage by memory representation.

Diagnostic Item	Verbatim	Summary	Reduction Percentage
Logical Provider Requests and Physical Attempts	288 logical and 290 physical	288 logical and 290 physical	No difference
Provider input tokens	3119459	1050382	66.33%
Output tokens	269498	262081	2.75%
Total tokens	3388957	1312463	61.27%
Median Provider Latency	12.07 s	12.29 s	Similar medians

Output tokens decreased by only 2.75%, so the main savings came from the input side.

Shorter context did not improve the observed response speed. The median request latency was 12.07 seconds for Verbatim and 12.29 seconds for Summary. Reducing input tokens therefore did not produce a shorter median latency in this experiment.

5.3.3 FAILURES AND RETRIES

There were five failures for each memory representation, for a total of ten. All ten stopped at the Validator with `ResponseFailure` and did not proceed to Authority commit. Five

returned `revise_plan`, and the other five returned malformed `verify`. The balanced failure count does not indicate a difference between the memory representations. The recorded outputs instead show failures in tool selection or parameter structure, although the experiment did not isolate their underlying cause.

Additionally, four requests returned `InvalidResponseEnvelope` on the first attempt: two for `Verbatim` and two for `Summary`. All four requests received a usable response on the second attempt. The time and tokens generated by retries are already included in the Provider usage above. No requests reached the output-token limit, and there were no instances of missing token usage.

5.4 COGNITIVE LOD AND LOCAL L1 EVALUATION

This section reports a supplementary evaluation of the system.

5.4.1 LOCAL L1 QUALITY AND COST

The local model produced an acceptable decision in 114 of 120 observations, or 95.00%. The remote model produced an acceptable decision in 119 of 120 observations, or 99.17%. The local-minus-remote difference was -4.17 percentage points. Its one-sided 95% lower bound was -7.50 percentage points, below the predeclared -5-point margin. The batch therefore did not demonstrate local-model non-inferiority.

TABLE 5.9: Local L1 decision quality.

Quality Measure	Result
Local acceptable decisions	114/120 (95.00%)
Remote acceptable decisions	119/120 (99.17%)
Local minus remote	-4.17 percentage points
One-sided 95% lower bound	-7.50 percentage points
Predeclared non-inferiority margin	-5.00 percentage points; criterion not met
Route agreement	113/120 (94.17%)
Exact agreement	83/120 (69.17%)

TABLE 5.10: Local L1 outcomes by expected route.

Case Group	Pairs	Local Acceptable	Remote Acceptable	Route Agreement
Choose	40	40/40	40/40	40/40
Defer	20	16/20	19/20	15/20
Request Escalation	60	58/60	60/60	58/60

Clear bounded choices were the strongest group: the local model completed all 40 Choose observations successfully. The weakest group was Defer, especially when the case contained insufficient evidence. These results indicate that the local model can handle clear, bounded choices, but ambiguous defer and boundary cases still need deterministic checks or a stronger model.

TABLE 5.11: Local and remote L1 Provider usage.

Usage Measure	Local Qwen3-8B	Remote DeepSeek
Input tokens	56290	3725 cache-miss plus 108416 cache-hit
Output tokens	62278	51703
Reported reasoning tokens	57052	Not returned separately
Mean latency per decision	8.03 s	6.13 s
Median latency per decision	4.73 s	4.90 s

The two Providers use different tokenizers, so their token counts are not directly interchangeable. At the recorded remote prices, using the local model for 1000 bounded L1 decisions avoids an estimated United States dollar (USD) 0.89 to USD 1.79 in remote Provider charges.

5.4.2 SIXTY-DAY WORKLOAD RESULTS

TABLE 5.12: Sixty-day Cognitive LOD workload.

Workload Measure	Days 1-30	Days 31-60	Days 1-60
L0 action receipts	181035	180978	362013
L0 completed actions	181035	180978	362013
L0 rejected actions	0	0	0
Autonomy L1 requests	6540	7182	13722
Mean L1 requests per NPC per ten configured Demo minutes	10.90	11.97	11.44
L1 Choose decisions	13	0	13
L1 Defer decisions	6495	7182	13677
L1 escalation requests	0	0	0
L1 failures	32	0	32
Autonomy L2 items recorded but not dispatched	120	120	240

Across all 25 NPCs, the cumulative median was 9.46 L1 requests per NPC per ten configured Demo minutes, ranging from 2.38 to 24.17. The mean increased from 10.90

during days 1-30 to 11.97 during days 31-60.

5.4.3 DIALOGUE TIERING RESULTS

The dialogue results for both the natural 60-day workload and the controlled routing suite are reported separately.

TABLE 5.13: Natural dialogue workload.

Natural Dialogue Measure	Sixty-Day Result
L0 invitation settlements	0
LocalReply outcomes	66
L1 escalation requests	29
Host-accepted escalation requests	29
Host-rejected escalation requests	0
Logical L2 dispatches	29
Terminally completed responses	95
Terminal failures by stage	145 at L1 contract
Local Provider tokens and latency	266875 tokens; median 7.82 s
Remote Provider tokens and latency	85481 tokens; median 7.30 s

The natural workload contained 240 `CasualComment` response opportunities and zero L0 invitation observations. Of these, 66 ended in `LocalReply`, and 29 created valid escalation requests. The Host accepted all 29 requests and dispatched them to L2, where they ended in `StrategicReply`. Another 145 response opportunities failed the L1 contract. The local dialogue model was called 240 times, reported using 266875 tokens, and had a median latency of 7.82 seconds. The remote model was called 29 times, reported using 85481 tokens, and had a median latency of 7.30 seconds. Since 145 response opportunities failed at L1 rather than completing locally, the 29/240 remote dispatch rate cannot be interpreted as a quality-preserving cost saving.

The controlled suite achieved an expected-route match in 175 of 180 observations and an acceptable terminal outcome in 155 of 180. L0 matched the expected route and reached an acceptable outcome in all 60 observations. L1 matched all 60 routes but reached an acceptable outcome in 58. L2 matched 55 of 60 routes and reached the strict acceptable terminal outcome in 37.

Among the 60 expected-L2 observations, 14 produced a `StrategicReply` under a different escalation-reason label, four ended in `RemotePlannerFailure`, and five did not leave L1 because the local output violated its contract.

TABLE 5.14: Controlled dialogue routing outcomes.

Expected Route	Cases	Observations	Expected-Route Match	Acceptable Outcome	Key Error Rate
L0	12	60	60/60	60/60	Invitation instability: 0/60
L1	12	60	60/60	58/60	Local contract failure: 2/60
L2	12	60	55/60	37/60	Missed escalation: 5/60
All	36	180	175/180	155/180	Reported separately by route

DISCUSSION

This chapter first discusses the implications of RQ1 and RQ2, then considers Cognitive LOD, and ends with system implications and limits.

6.1 INTERPRETATION OF RQ1

RQ1 did not reveal a preferred admission policy based on grounded terminal success. EventCentric achieved 20.00% success and AgentCentric achieved 19.00% success, resulting in an EventCentric minus AgentCentric difference of 1.00 percentage point. The paired interval was from -4.00 to $+5.67$ percentage points, and the exact sign test yielded $p = 0.38331$. Under the specified 15-point threshold, neither policy outperformed the other by a practically meaningful amount within the evaluated blocks.

The 30-block design varied admission choices across distinct blocks. Seventy-five percent of each policy's admissions were exclusive, and the success differences remained small across the zero-, one-, and two-overlap tiers. Among the 30 shared admissions, both policies used identical request inputs and achieved 13 successes. The discordant outcomes for these admissions were also balanced; four successes were unique to EventCentric and four successes were unique to AgentCentric. The overall difference of three successful terminals arose among policy-exclusive cases.

The failure-stage analysis provided the clearer engineering result. All 123 unsuccessful admitted sessions ended at the Validator. The most common pattern was a Host-accepted CreatePlan that did not reach a grounded endpoint. There were 74 such instances, all occurring in cases where the expected result was a justified defer. Expected-commit cases succeeded in 64.17% of admitted sessions, while expected-defer cases succeeded in 33.33% of admitted sessions. The same pattern was observed under both admission policies.

Event-centred planning and influence-based abstraction are based upon the relationships between events, systems, and affected agents (Shoulson et al. 2013; Oliehoek, Witwicki, and Kaelbling 2012). These relationships make it possible to determine who is affected by an event; however, they do not establish whether the current planning contract has a grounded path for resolving the need. Therefore, dependency impact and resolution readiness are two separate admission signals.

The results of RQ1 showed no differences in the single-window fairness diagnostics. Each policy admitted four distinct Actors; therefore, sixty percent of the eligible pressure Actors were left without a slot. The 95th percentile (P95) wait among admitted Actors was

one tick and `StarvationAge` was not triggered. Cognitive LOD work takes into account both behavioural value and processing cost (Niederberger and Gross 2005; Wißner, Kistler, and André 2010; Navarro, Flacher, and Corruble 2011). These results also show that allocating the same number of expensive processing sessions does not guarantee equal or successful terminal outcomes.

6.2 INTERPRETATION OF RQ2

The RQ2 results are more definite. Verbatim and Summary started from identical candidate sets and orders. The experiment varied only how each set was rendered into a `MemoryPacket`; memory retrieval did not differ. Summary compresses multiple source records into less content with source identifiers while retaining sufficient evidence for the action. Each memory representation achieved grounded world-action success in 283 of 288 runs. Summary’s success-rate difference relative to Verbatim is 0, with a one-sided lower bound of -1.74 percentage points, exceeding the non-inferiority threshold of -5 percentage points.

Compared to memory-based methods that primarily utilize reflection text or relevant fragment retrieval (Park et al. 2023; Gutiérrez et al. 2024), the Summary presented herein also retains the source identifier and information supersession relationships. The system can verify which Actor’s experience produced a given statement in the summary and verify if older messages have been replaced by newer events. The summary here is not an independently generated background summary, but rather a decision input that allows for source tracking.

Summary reduces the total number of estimated `MemoryPacket` tokens by 98.51%. However, Provider input tokens only decrease by 66.33%. This occurs because the complete request also includes system prompts, tool definitions, and the current actor-visible state, all of which remain unaltered with memory compression. Therefore, 98.51% represents a reduction in the memory portion, whereas 66.33% is more indicative of actual changes to input received by the model.

These savings cover the decision stage only. The Summary condition used 36 source-linked artifacts that were generated before formal collection. Each artifact was reused across the eight model repeats in its stratum-tier cell. The 36 generation requests were preparation evidence. They were not RQ2 observations.

The reported 98.51% `MemoryPacket` reduction and 66.33% Provider-input reduction do not include the cost of generating those artifacts. They also do not measure how Summary generation might vary across repeated generations. The cost result therefore applies after a valid Summary artifact already exists. It is not an estimate of the full cost of a live summary-generation pipeline.

Output tokens decrease by only 2.75%, and median latency for Verbatim and Summary are nearly identical. Research concerning long-term memory frequently addresses whether a model can recall information from an extended history (Maharana et al. 2024; Wu et al. 2025). This study demonstrates that source-linked summaries can substantially reduce

input, yet do not necessarily result in reduced content generated by the model, nor do they necessarily reduce Provider latency. Input length, output length, and Provider latency must be addressed independently.

This high success rate does not invalidate the non-inferiority test, which was designed to ask whether Summary stayed within five percentage points of Verbatim when the two success rates were close.

The task included stale state, conflicting reports and other cases designed to challenge the memory representation. Ten condition outcomes were unsuccessful. All ten were stopped at the Validator due to tool choice or parameter structure. Five results returned `revise_plan`, and five results returned malformed `verify`. None was traced back to a missing candidate record, a missing required source, or a broken Summary source link. The observed failures, therefore, do not demonstrate that memory compression caused a quality loss. The observed failures indicate that the recorded failure stage was the tool interface; this is consistent with the distinction between information access and correct tool execution as documented in tau-bench (Yao et al. 2025).

The supported claim remains specific. These 36 prepared Summary artifacts passed the source-link checks and were non-inferior to Verbatim on this task set. The study does not show that every summary-generation pipeline will preserve the same quality.

6.3 COMBINED SYSTEM IMPLICATIONS

RQ1 and RQ2 address different runtime decisions. RQ1 determines which needs enter L2, whereas RQ2 determines how an admitted decision receives its historical context. RQ1 does not support selecting either admission policy as the default on grounded-success evidence. The shared failure pattern instead motivates a treatment-neutral check of whether an admitted need has an executable, verifiable or deferable path. After admission, RQ2 provides support for reducing the memory context that is presented to the model through the use of a source-linked Summary.

6.3.1 A RESOLUTIONREADINESS SIGNAL FOR ADMISSION

A specific design change follows from the shared failure pattern. `DependencyIndex` identifies Actors affected by an event, whereas a separate `ResolutionReadiness` signal would describe what the current planning and action contract can do for those Actors.

The proposed signal returns `ExecutableNow`, `VerificationRequired`, `PlanOnly` or `NoGroundedPath`. Needs that do not have an immediate grounded path could then be routed towards verification or a justified defer. The same signal should be available to both admission policies. Chapter B describes this design; however, it was not evaluated in the present experiment.

6.4 PRACTICAL VALUE OF L0 AND LOCAL L1

The Cognitive LOD supplement asks whether the implemented levels have practical roles outside the two core experiments. RQ1 and RQ2 study decisions at the L2 boundary, but they do not show how much routine work is completed before L2 is needed.

The local-model comparison gives limited support for the practical value of L1. The local model returned an acceptable decision in 95.00% of the bounded observations and completed all 40 clear Choose observations successfully. It also agreed with the remote model's main route in 94.17% of observations. These results show that a local model can handle many clear and narrow decisions while avoiding a paid remote call.

However, the local model did not pass the predeclared non-inferiority requirement. Its one-sided lower bound was -7.50 percentage points, below the -5-point margin, and its main weakness was *Defer* under insufficient evidence. It therefore cannot be described as a general replacement for the remote model. The stronger interpretation is that the local model is useful for clear bounded choices when rule checks or a stronger path remain available for ambiguous *defer* and boundary cases.

The estimated USD 0.89 to USD 1.79 avoided remote charge per 1000 L1 decisions shows the scale of the Provider-side saving in this batch.

The 60-day workload reflects that a significant portion of the routine work in this town was executed by L0. L0 produced 362013 completed action receipts (without using models) and recorded no rejected actions. The autonomy system produced 13722 bounded L1 requests at an average rate of 11.44 requests per NPC every ten minutes and recorded 240 L2 items without dispatch. Since these statistics represent different units, it is not appropriate to convert them into a single percentage representing the work handled by L0, L1, and L2. The autonomy L1 branch used deterministic settlement; therefore, the 60-day trajectory is not able to reflect how the world would develop after 13722 real local-model decisions. It only measures how often the current town produces bounded L1 work.

The controlled dialogue suite achieved an expected-route match in 175 of 180 observations. L0 successfully handled all 60 clear invitation observations without a model call. L1 successfully handled 58 of 60 ordinary response observations. In the 60 expected-L2 observations, 55 reached L2; however, only 37 reached the strict acceptable terminal outcome. This shows that correct routing does not guarantee that the final response will be successful. All contract-valid escalation requests in the controlled suite were accepted by the Host.

This batch demonstrates that the majority of designed strategic cases can reach L2, but it does not separately demonstrate that the Host would reject invalid ordinary escalations. Of 240 response opportunities generated by the natural workload, 66 ended in *LocalReply* and 29 ended in *StrategicReply*.

Additionally, 145 response opportunities failed at the L1 contract. The primary issue with this workload was therefore the contract itself, not Provider transport or Authority settlement. There were no NPC invitations in the natural workload, so it cannot be used to estimate the natural frequency of L0 invitation handling.

6.5 THREATS TO VALIDITY

The RQ1 experiment utilized one town of 25 NPCs, one typed action catalogue and one remote model. The tasks associated with the blocks used in RQ1 spanned a variety of task families, expected terminal types, and admission-overlap levels. As such, the variety of scheduling decisions was greater than if multiple iterations of a single pressure set were used. However, all of the blocks were designed for the same runtime and action contract.

The end-to-end effect of each admission policy is measured by RQ1. The use of shared admissions enables comparisons between different admission policies based upon identical request inputs. Policy-exclusive admissions reflect the tasks that each ranking method actually selects. Since exclusive tasks may vary in terms of downstream difficulty, the RQ1 experiment does not separate ranking-signal quality from task resolvability. Therefore, RQ1 draws conclusions about the complete AgentCentric and EventCentric policies under the tested runtime.

RQ2's 288 matched pairs originate from 36 stratum-tier cells, with eight model repeats per cell. Since each cell reuses the same source-linked Summary across eight repeats, this experiment assesses performance of these summaries as decision inputs without assessing changes from regenerated summaries. Conclusions cannot be directly generalized to knowledge graphs, retrieval augmentation, or other summarization techniques. Systems such as AriGraph utilize distinct memory structures (Anokhin et al. 2025), requiring further comparisons.

The estimator is `utf8_bytes_div4_v1`. It estimates `MemoryPacket` tokens and is not the Provider's own tokenizer. Consequently, a 98.51% reduction cannot be directly interpreted as a 98.51% reduction in billing. Provider usage records show a 66.33% reduction in input utilization, illustrating that compression impacts actual requests. However, these two percentages do not relate to the same component.

Stress cases, required sources, task endpoints, and acceptable actions were all defined by researchers. A consistent scoring method reduces variability in evaluations between two processes, but it cannot eliminate influence of case selection. Additionally, this paper evaluates whether actions are executed correctly, not whether NPCs are natural, interesting, or life-like. An action that passes rule checks may still bore players. Conversely, a reasonable deferral may be interpreted as an unresponsive NPC. Task success rate cannot substitute for a study of player interactions.

There are 24 unique cases in this local L1 comparison, with each case repeated 5 times. These repetitions show how the different models vary, however they do not represent 120 independent problem types.

Additionally, this analysis was performed using the configured Qwen3-8B and DeepSeek models and the previously specified bounded L1 cases. The 60-day workload is a single continuous trajectory that has two distinct 30-day windows which serve as descriptive summaries of the workload, and thus cannot be considered two separate experimental groups. Autonomy level 1 utilized deterministic settlement while autonomy level 2 items

were recorded without dispatch, so the resulting counts reflect workload generation and not a live model-shaped world trajectory. The controlled dialogue suite includes an intentionally balanced sample of L0, L1, and L2 cases, so the proportion of routes it represents does not reflect the natural frequency of dialogue levels found throughout the town.

6.6 LIMITATIONS AND FUTURE WORK

Each block in RQ1 has one 240-tick budget window. Due to this limitation, RQ1 cannot evaluate long-term fairness. The *StarvationAge* rule requires unresolved needs to remain queued across multiple scheduling points, which was outside the current experiment. In order to assess long-term fairness, a longitudinal study would be required. Such a study would track the same unresolved *DecisionNeeds* through subsequent budget windows and monitor changes in admission order over time.

The Living Town Demo shows that persistent states, memories, actions, dialogue, and saves can be combined in one runtime. However, dialogue and random behaviours observed only in the Demo were not included in RQ1 or RQ2. Other models, towns, and summary-generation techniques still require separate testing.

The natural dialogue workload showed a discrepancy between the local model's output and the L1 contract. Future research should simplify or clarify the contract and repeat the natural dialogue evaluation.

7 CONCLUSION

This chapter answers the research questions and closes the thesis.

7.1 RESEARCH SUMMARY

This paper establishes a limited cognitive runtime for persistent NPCs. Persistent NPCs maintain their Actor states even after exiting player’s line-of-sight. Daily activities are handled by L0 and L1, and issues needing replanning are stored as `DecisionNeeds Queued` for logical L2 sessions. Upon securing a slot, the system constructs `MemoryPacket` from actor-visible history. Host controls escalation, L2 dispatch, and proposal checks. Validator checks concrete actions, and only Authority commits world-state changes.

7.2 ANSWERS TO THE RESEARCH QUESTIONS

RQ1 compared the selection of four `DecisionNeeds` by `AgentCentric` and `EventCentric` admission policies under a fixed $B = 4$ budget. Each admission policy was evaluated using ten simultaneous pressure cases. `AgentCentric` reached 57 correct grounded terminals out of 300 possible opportunities, a rate of 19.00%, over 30 distinct blocks. `EventCentric` reached 60 correct grounded terminals, a rate of 20.00%. The `EventCentric` minus `AgentCentric` difference was 1.00 percentage point. A 95% paired interval for the difference was -4.00 to $+5.67$ percentage points. `EventCentric` performed better in 13 blocks, `AgentCentric` performed better in 8 blocks and 9 blocks were tied. Using an exact sign test, the resulting value was $p = 0.38331$. Therefore, neither policy outperformed the other by the specified 15-point practically meaningful amount.

The policies differed on 75% of their admissions; however, they had equal success counts on the 30 shared admissions. Structural acceptance of a plan followed by failure to reach the expected grounded terminal represented the most common unsuccessful outcome for both policies. This failure pattern motivates separating dependency impact from resolution readiness, rather than selecting either admission policy solely on the basis of success rate.

The second question (RQ2) compared `Verbatim` and `Summary`, both using the same candidate sets and an 8192-estimated-token memory-context ceiling. Each memory representation achieved grounded world-action success in 283 of 288 runs. The `Summary`-minus-`Verbatim` success-rate difference was 0.00 percentage points. A one-sided 95% bootstrap

lower bound on the difference was -1.74 percentage points. This value was above the predeclared non-inferiority margin of -5 percentage points.

Thus, Summary met the non-inferiority criterion on this task set. It also decreased the estimated MemoryPacket token count by 98.51% and the actual Provider input token count by 66.33%. No continuous decline in task success was observed across the six history-load tiers. The token reduction percentages listed above are for the decision stage of the process only and do not include the 36 preparation calls used to generate the frozen Summary artifacts.

7.3 SUPPLEMENTARY COGNITIVE LOD FINDINGS

The remote DeepSeek returned an acceptable decision in 119 of the 120 observations, whereas the local Qwen3-8B returned an acceptable decision in 114 of the 120 observations. All 40 clear Choose observations were completed successfully by the local model. However, its one-sided 95% lower bound was -7.50 percentage points, which was below the specified -5 percentage point non-inferiority margin. Consequently, although the local model could manage many clear and bounded choices, it could not substitute for the remote model for all L1 cases. Based on the Provider prices utilized in this study, every 1000 local L1 decisions can avoid an estimated USD 0.89 to USD 1.79 in remote Provider charges.

During the 60-day period of workload, L0 performed 362013 routine actions without making any calls to a model, and none of those actions were rejected. Autonomy created 13722 bounded L1 requests and recorded 240 L2 items without dispatch. The data indicates that L0 has the ability to perform many routine world tasks. Nevertheless, the autonomy L1 requests were settled using deterministic rules. Consequently, this workload measures how frequently L1 work is produced. It does not show how the world would be altered after 13722 live local-model decisions.

The controlled dialogue suite achieved an expected-route match in 175 of 180 observations and an acceptable terminal outcome in 155. In the natural workload, 66 response opportunities ended in LocalReply and 29 created valid escalation requests. Each request was accepted by the Host and dispatched to L2, but another 145 response opportunities failed at the L1 contract. The controlled suite shows that the routing design can handle most prepared cases, while the natural workload shows that the current L1 contract is not yet reliable.

7.4 FINAL REMARKS

Therefore, these results indicate that persistent NPCs do not require a remote model for every routine action nor do they need to incorporate their complete history into every L2 decision. The system can first determine which needs merit L2 admission and then assign shorter, source-linked memory representations to those needs. RQ1 revealed no practically meaningful success advantage for either AgentCentric or EventCentric admission policies

in the blocks tested. RQ2 demonstrates that source-linked Summary can reduce context substantially while achieving similar grounded world-action success rates.

L0 is capable of completing a significant quantity of deterministic work, and the local model can support clear bounded decisions; however, the L1 contract still requires improvement.

BIBLIOGRAPHY

- Anokhin, Pavel et al. (2025). “AriGraph: Learning Knowledge Graph World Models with Episodic Memory for LLM Agents”. In: *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence*, pp. 12–20. DOI: 10.24963/ijcai.2025/2.
- Chen, Lingjiao, Matei Zaharia, and James Zou (2024). “FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance”. In: *Transactions on Machine Learning Research*. ISSN: 2835-8856. URL: <https://openreview.net/forum?id=cSimKw5p6R>.
- Flores, Luis and David Thue (2017). “Level of Detail Event Generation”. In: *Interactive Storytelling: ICIDS 2017*. Springer, pp. 75–86. DOI: 10.1007/978-3-319-71027-3_7.
- Gutiérrez, Bernal Jiménez et al. (2024). “HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models”. In: *Advances in Neural Information Processing Systems*. Vol. 37. URL: https://proceedings.neurips.cc/paper_files/paper/2024/hash/6ddc001d07ca4f319af96a3024f6dbd1-Abstract.html.
- Liu, Nelson F. et al. (2024). “Lost in the Middle: How Language Models Use Long Contexts”. In: *Transactions of the Association for Computational Linguistics* 12, pp. 157–173. DOI: 10.1162/tac1_a_00638.
- Maharana, Adyasha et al. (2024). “Evaluating Very Long-Term Conversational Memory of LLM Agents”. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pp. 13851–13870. DOI: 10.18653/v1/2024.acl-long.747.
- Minsky, Marvin (1980). “K-Lines: A Theory of Memory”. In: *Cognitive Science* 4.2, pp. 117–133. DOI: 10.1207/s15516709cog0402_1.
- Morvan, Gildas, Antoine Veremme, and Daniel Dupont (2011). “IRM4MLS: The Influence Reaction Model for Multi-Level Simulation”. In: *Multi-Agent-Based Simulation XI*. Springer, pp. 16–27. DOI: 10.1007/978-3-642-18345-4_2.
- Navarro, Laurent, Fabien Flacher, and Vincent Corruble (2011). “Dynamic Level of Detail for Large Scale Agent-Based Urban Simulations”. In: *Proceedings of the 10th International Conference on Autonomous Agents and Multiagent Systems*, pp. 701–708. URL: https://aamas.csc.liv.ac.uk/Proceedings/aamas2011/papers/C5_B67.pdf.
- Niederberger, Christoph and Markus Gross (2005). “Level-of-Detail for Cognitive Real-Time Characters”. In: *The Visual Computer* 21.3, pp. 188–202. DOI: 10.1007/s00371-005-0279-1.

- Oliehoek, Frans A., Stefan J. Witwicki, and Leslie P. Kaelbling (2012). “Influence-Based Abstraction for Multiagent Systems”. In: *Proceedings of the Twenty-Sixth AAAI Conference on Artificial Intelligence*, pp. 1422–1428. DOI: 10.1609/aaai.v26i1.8253.
- Ong, Isaac et al. (2025). “RouteLLM: Learning to Route LLMs from Preference Data”. In: *International Conference on Learning Representations*. URL: https://proceedings.iclr.cc/paper_files/paper/2025/hash/5503a7c69d48a2f86fc00b3dc09de686-Abstract-Conference.html.
- Park, Joon Sung et al. (2023). “Generative Agents: Interactive Simulacra of Human Behavior”. In: *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. Association for Computing Machinery, pp. 1–22. DOI: 10.1145/3586183.3606763.
- Poulakos, Steven et al. (2015). “Towards an Accessible Interface for Story World Building”. In: *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*. Vol. 11. 4, pp. 42–48. DOI: 10.1609/aiide.v11i4.12833.
- Shoulson, Alexander et al. (2013). “An Event-Centric Planning Approach for Dynamic Real-Time Narrative”. In: *Proceedings of Motion on Games*, pp. 121–130. DOI: 10.1145/2522628.2522629.
- Sunshine-Hill, Ben (2013). “Managing Simulation Level-of-Detail with the LOD Trader”. In: *Proceedings of Motion on Games*, pp. 13–18. DOI: 10.1145/2522628.2541250.
- Vezhnevets, Alexander Sasha et al. (2023). *Generative Agent-Based Modeling with Actions Grounded in Physical, Social, or Digital Space Using Concordia*. arXiv: 2312.03664 [cs.AI]. URL: <https://arxiv.org/abs/2312.03664>.
- Wißner, Michael, Felix Kistler, and Elisabeth André (2010). “Level of Detail AI for Virtual Characters in Games and Simulation”. In: *Motion in Games 2010*. Springer, pp. 206–217. DOI: 10.1007/978-3-642-16958-8_20.
- Wu, Di et al. (2025). “LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory”. In: *International Conference on Learning Representations*. URL: <https://openreview.net/forum?id=pZiyCaVuti>.
- Yao, Shunyu et al. (2025). “tau-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains”. In: *International Conference on Learning Representations*. URL: <https://openreview.net/forum?id=roNSXZpUDN>.

SUPPLEMENTARY COGNITIVE LOD

MATERIALS

This appendix records the compact evidence structure for the supplementary Cognitive LOD evaluation. The supplement is separate from RQ1 and RQ2.

A.1 LIFECYCLE LOG FIELD DICTIONARY

The dialogue harness writes one terminal lifecycle record for each response opportunity. It stores identifiers and compact results instead of copying full prompts, responses, or world snapshots into every row.

TABLE A.1: Dialogue lifecycle fields.

Field Group	Recorded Fields
Run identity	Run ID, suite or workload identity, opportunity ID, and optional controlled case ID
Simulation context	Simulation day and time, responding Actor ID, source Actor ID, source act kind, and source event ID
L0 route	L0 eligibility, terminal outcome, and reason
L1 route	Invocation status, typed decision, output validity, failure kind, tokens, latency, and attempt count
Escalation	Request status, reason code, actor-visible evidence reference, and Host result
L2 route	DecisionNeed ID, logical session ID, Provider-dispatch status, and terminal resolution
World result	Final semantic response kind and Authority receipt ID when a world consequence is committed
Usage audit	Model role, input, output, reasoning, cached and total tokens, latency, retry count, and usage-known flag

Full credential-free Provider payloads, when needed for audit, remain in sidecar artifacts referenced by ID. Provider thinking content is not copied into this appendix.

A.2 CONTROLLED DIALOGUE SCENARIO LIST

The controlled suite contains 36 distinct actor-visible cases. The expected route is frozen separately from the case input. Each case was repeated five times.

TABLE A.2: Controlled Cognitive LOD dialogue scenarios.

Case ID	Expected Result	Actor-Visible Scenario
L0-A01	L0 Accept	Alice invites Bram to the evening gathering.
L0-A02	L0 Accept	Celia invites Devin to a public meal.
L0-A03	L0 Accept	Emma invites Felix to the market gathering.
L0-A04	L0 Accept	Grace invites Hugo to a neighbourhood supper.
L0-A05	L0 Accept	Iris invites Jonah to a town-square meeting.
L0-A06	L0 Accept	Kara invites Lucas to a routine community gathering.
L0-D01	L0 Decline	Mira invites Nolan to an optional evening gathering.
L0-D02	L0 Decline	Opal invites Pavel to an optional public meal.
L0-D03	L0 Decline	Quinn invites Rosa to a routine market gathering.
L0-D04	L0 Decline	Silas invites Aiko to an optional social visit.
L0-D05	L0 Decline	Bernard invites Clara to a routine evening meal.
L0-D06	L0 Decline	Dorian invites Elin to an optional town-square gathering.
L1-01	L1 LocalReply	Good morning. The weather feels calm today.
L1-02	L1 LocalReply	How is your work going this morning?
L1-03	L1 LocalReply	The market is busy today.
L1-04	L1 LocalReply	Which local meal would you recommend?
L1-05	L1 LocalReply	It is good to see familiar people in town.
L1-06	L1 LocalReply	I am tired after today's work.
L1-07	L1 LocalReply	Do you think I should rest before supper?
L1-08	L1 LocalReply	I saw several people near the town square.
L1-09	L1 LocalReply	What are you doing right now?
L1-10	L1 LocalReply	The road looks quiet this afternoon.
L1-11	L1 LocalReply	Can you explain where the inn is?
L1-12	L1 LocalReply	I hope your day becomes easier.
L2-01	L2 StrategicReply	A resource commitment is requested when no feasible local action is available.
L2-02	L2 StrategicReply	A promise is requested for a shortage with no known feasible local action.
L2-03	L2 StrategicReply	A failed work plan requires a change to the long-term goal.
L2-04	L2 StrategicReply	The current plan must be abandoned and replaced with a new long-term plan.

Case ID	Expected Result	Actor-Visible Scenario
L2-05	L2 StrategicReply	A loan of five coins creates a repayment commitment.
L2-06	L2 StrategicReply	An existing tool debt requires a new repayment commitment.
L2-07	L2 StrategicReply	A betrayal leads to a request to end a partnership.
L2-08	L2 StrategicReply	A threat leads to a major change in trust and relationship.
L2-09	L2 StrategicReply	Medicine is required before a deadline to prevent greater harm.
L2-10	L2 StrategicReply	Worsening health requires treatment before a deadline.
L2-11	L2 StrategicReply	A work plan has failed three times and requires a different response.
L2-12	L2 StrategicReply	Repeated delivery failure requires a different strategic response.

A.3 REPRESENTATIVE LIFECYCLE RECORDS

The records below illustrate four representative terminal paths. These records only contain the field values necessary to describe how the route was determined.

- Successful L0 route: The CaseID is L0-A01. The RepeatNum is 1. The Responder is bram. The SourceAct is Invite. The Route is L0. The TerminalResult is Accept. There were no Provider requests.
- Successful L1 route: The CaseID is L1-01. The RepeatNum is 1. The Responder is bram. The Decision is choose. The Route is L1. The TerminalResult is LocalReply.
- Successful L2 route: The CaseID is L2-01. The RepeatNum is 1. The Responder is bram. The Decision is request_escalation. The Reason is no_feasible_local_action. The Host accepted the escalation request. The TerminalResult is StrategicReply.
- Natural L1 contract failure: On Day 1, iris received a CasualComment. Transport succeeded. However, the local model chose choose while classifying the incoming effect as Harm. As a result, the L1 Decoder returned invalid_structured_output.

The complete lifecycle records and Provider sidecars remain in the supplementary artifact package.

PROPOSED RESOLUTION READINESS DESIGN

This appendix expands the design sketch referenced in Section 6.3.1. `ResolutionReadiness` is a proposed deterministic signal. It is not part of the current implementation, and it was not tested in this study.

A specific design change follows from the shared failure pattern reported in Table 5.4: 74 of the 123 unsuccessful admitted sessions were a Host-accepted `CreatePlan` that never reached a grounded endpoint. `DependencyIndex` answers one question: who is affected by an event? It does not answer a second question: what can the current L2 planning contract do for that Actor now? These questions should use separate signals. `ResolutionReadiness` describes the next useful step that this contract can support. The signal would use the following inputs:

- the eligible `DecisionNeed` and its actor-visible `ProblemDescriptor`;
- the Actor's visible goal and current plan state;
- the typed L2 tool catalogue and its legal `PlanStep` pairings;
- item, resource, target, and opportunity references already visible to the Actor or available to the deterministic Host.

The signal must not read the hidden scorer, the expected experimental answer, the treatment label, or Authority-only truth that the Actor cannot know.

The current L2 proposal schema does not expose a concrete `GameActionSpec`. The signal therefore cannot be inferred from an objective name alone. A reliable implementation needs a small Host-owned capability map. This map would connect each legal objective and desired-result pair to the Authority endpoint types that can ground it. It would also list the visible references required for that path. Without this map, an abstract `experience_completed` step could be mistaken for an executable path simply because its words match the goal.

The signal would return one of four states. It would also return a short reason code and the references used for the decision.

TABLE B.1: ResolutionReadiness states.

State	Meaning
ExecutableNow	The L2 planning contract exposes a typed route to a grounded endpoint, and the required visible references resolve.
VerificationRequired	A grounded proposal may exist, but a specific visible fact must be checked first through verify.
PlanOnly	The L2 planning contract can create or revise a valid plan, but it cannot reach the task endpoint in the present session.
NoGroundedPath	The L2 tool and action catalogues, with the visible state, do not expose a grounded path to the requested endpoint.

The proposed flow is:

DependencyIndex

-> EventCentric impact rank: E0 / E1 / E2

-> ResolutionReadiness

ExecutableNow

VerificationRequired

PlanOnly

NoGroundedPath

-> admission order

ResolutionReadiness should not silently remove a need. A reasoned defer is a valid outcome in the current study. A need can also become resolvable after new evidence or a world-state change. The state should guide routing and ranking. It should not act as a permanent rejection. The shared StarvationAge rule should remain in place.

One concrete follow-up policy could keep mandatory responses and deadline urgency first. Within the same urgency band, it could rank readiness before the EventCentric E0/E1/E2 impact band. The remaining age and stable tie-break rules would stay in place. This ordering is a design hypothesis. The current results do not show that it is the best ordering.

mandatory response

-> deadline urgency

-> ResolutionReadiness

-> E0 / E1 / E2

-> age and stable tie-breaks

If ResolutionReadiness is added only to EventCentric, the result is a new hybrid treatment. It should be named EventCentric+Readiness. Its results must not be reported as if they came from the EventCentric policy used in this thesis.

There are two fair ways to test the idea. A direct ablation can compare EventCentric with EventCentric+Readiness. This tests whether the new signal helps EventCentric. A second experiment can give the same ResolutionReadiness signal to both admission

policies. This tests AgentCentric and EventCentric under a shared ResolutionReadiness layer. Neither experiment has been run here.

REPRODUCIBILITY AND SUPPLEMENTARY RESOURCES

This appendix links to the external resources that accompany this dissertation: the source code repository, a recorded demonstration of the Living Town Demo, and the research artifact bundles referenced throughout Chapters 4 and 5. These resources support inspection of the prototype, the available experiment fixtures, and the reported outcomes. They do not include the hidden scoring materials described in Section 4.6.

C.1 SOURCE CODE

The public repository contains the runtime and formal-study harness released with this dissertation:

<https://github.com/Away1615/llm-rpg>

C.2 DEMONSTRATION VIDEO

A recorded walkthrough of the Living Town Demo (Figure 3.3), showing L0, L1, and L2 cognition routing during an interactive session, is available at:

https://drive.google.com/file/d/1IAQXfz1sI1Py_1K-T4gXJvlofaCGY0Po/view

C.3 RESEARCH ARTIFACTS

The research artifact bundles for RQ1, RQ2, the local L1 comparison, and the supplementary Cognitive LOD evaluation are available at:

https://drive.google.com/file/d/1FAchR2JMORPUxCM6ZQfddlPAo0_Lz-tU/view?usp=sharing